HNSecW: A Logarithmic-Time MPC Protocol for Approximate Nearest Neighbor Search

Fahong Zhang
Nanyang Technological University
Singapore, Singapore
fahong001@e.ntu.edu.sg

Weijie Jin
Beihang University
Beijing, China
jinweijie@buaa.edu.cn

Lixu Wang
Nanyang Technological University
Singapore, Singapore
lixu.wang@ntu.edu.sg

Ning Luo
University of Illinois
Urbana-Champaign
Urbana, USA
nl27@illinois.edu

Xiao Wang
Northwestern University
Evanston, USA
wangxiao@northwestern.edu

Wei Dong
Nanyang Technological University
Singapore, Singapore
wei_dong@ntu.edu.sg

ABSTRACT

As a core component of modern RAG systems, approximate nearest neighbor (ANN) search has been extensively studied over the past decades. In practice, the deployment of ANN often requires symmetric privacy, where the querier's query and the database owner's data must remain hidden from each other, necessitating a secure multi-party computation (MPC) solution. So far, most MPC-based ANN protocols follow a cluster-based approach: they evaluate distances for the query over a large candidate set and rely on heavyweight primitives such as distributed ORAM to hide access patterns. As a result, each search typically requires $O(\sqrt{N})$ time. In this paper, we propose HNSecW, a novel MPC protocol for ANN search that achieves the $O(\log N)$ search time for the first time. We first study the single-query setting. Starting from HNSW, the state-of-the-art ANN algorithm, we redesign its control flow and data structures to achieve fully data-independent execution. To further achieve logarithmic search time, we employ offline reshuffling and partial database duplication. Furthermore, we extend our protocol to support multiple sequential queries by introducing a "double visited" caching structure that eliminates redundant cross-query memory accesses, thereby avoiding per-query online shuffling. In addition, we develop a batch-query protocol that exploits higher parallelism to reduce both round complexity and the circuit's critical path. Our experiments show that HNSecW preserves the high accuracy of plaintext HNSW ($\geq$ 90% recall) and delivers more than a 20× efficiency improvement over prior work on million-scale datasets.

CCS CONCEPTS

• **Security and privacy → Management and querying of encrypted data**.

KEYWORDS

Secure indexing, HNSW, Oblivious RAM, Multi-party computation, Vector search

1 INTRODUCTION

Efficient approximate nearest neighbor (ANN) search [21] over vector databases is a basic data analytical primitive and becomes a cornerstone of modern AI applications such as Retrieval-Augmented Generation (RAG) [26], which enhances Large Language Models (LLMs) by incorporating external knowledge at inference time. So far, designing an efficient ANN algorithm for a large-sized high-dimensional database has been well studied in the past several decades [28, 29, 42].

However, in real-world ANN applications, security and privacy concerns arise [6, 30, 48]. In practical settings, two distinct and mutually untrusting parties are always involved: a database owner and a querier. Without a protective protocol, either the querier submits the query in plaintext to the database owner, who executes the ANN algorithm, or the computation is delegated to the querier, which typically requires the database owner to reveal (at least part of) its data to enable search. The former violates query privacy, whereas the latter compromises database privacy. This motivates a symmetric private ANN model that simultaneously protects both the query and the database. We illustrate the need for symmetric privacy through two representative scenarios:

Example 1.1. In a joint healthcare study, a research institute (querier) prepares a drug trial by querying a hospital's (database owner) private gene-expression database for similar patient profiles. The hospital returns only the top records allowed by the data-use agreement, and the institute uses them as context for a large language model to generate a trial plan. During the collaboration, if the query is disclosed, it may reveal the institute's research details; if the hospital's database and patient data are disclosed, it would violate regulations such as HIPAA.

Example 1.2. During a cyber incident, a security team (querier) queries a vendor's (database owner) private threat intelligence database to retrieve the closest matching attack reports and indicators, and uses the results to identify the threat and respond promptly. If the query is disclosed, it can reveal what systems were hit; if the database is disclosed, it can expose sensitive intelligence sources and customer incidents.

A naive approach to achieving symmetric privacy is to introduce a trusted third party, such as a cloud provider: both the database owner and the querier submit their plaintext vector database and queries to the third party, which then performs the ANN search and returns only the results to the querier. However, delegating all sensitive information in plaintext to a single, fully trusted entity may violate privacy regulations like GDPR and introduce severe risks of data breaches or misuse.

This motivates people to seek cryptographic solutions that can remove the need for trusted third parties. One natural solution is

to implement ANN search in the secure multi-party computation (MPC) model [47]. In these solutions, the database owner and the querier secret-share the vector database and queries across multiple servers. The servers then collaboratively execute the ANN search protocol on these shares. The security of MPC ensures that no single server can reconstruct any input. Recently, we have seen several efforts devoted to this direction. However, most existing solutions either relax the privacy guarantees, i.e., assuming a public database [49], or incur high computational overhead [11, 27]. More specifically, the closest two MPC solutions to our setting [11, 27] both rely on cluster-based ANN pipelines: the database of size N is partitioned into several clusters, and the protocol first identifies the clusters most relevant to a given query, followed by a search within these clusters. The efficiency of such solutions depends heavily on the number of clusters, which is typically set to $\sqrt{N}$ to balance the cost of cluster selection and in-cluster search. As a result, their search time becomes $O(\sqrt{N})$ for each query. This quickly becomes impractical at modern database scales, because under MPC, each additional vector access incurs heavy communication, particularly for high-dimensional vectors. Earlier secure kNN works [32, 34, 36, 52] target smaller-scale or specialized two-party settings rather than scalable secret-shared ANN.

This motivates us to develop a more efficient MPC ANN protocol. The most natural idea is to deploy a state-of-the-art hierarchical graph-based ANN algorithm, such as HNSW [31], under the MPC setting. By building a multi-layer graph index over the database, such algorithms enable a very efficient search: the query starts at the top layer, takes only a few steps per layer, and then moves to lower layers. It is widely accepted that, in practice, these indexes visit only $O(\log N)$ out of N vectors per query in expectation, where N is the database size.

However, implementing this hierarchical graph-based search in MPC is very challenging. At a high level, we must make two parts of the algorithm oblivious: (i) the search logic, e.g., when to stop the search, and (ii) the access pattern. The stopping condition in HNSW is highly data- and query-dependent and replacing it with data-independent predicates without careful consideration totally compromises efficiency. For the access pattern, the search process may revisit the same node or vector multiple times, creating highly non-uniform and query-dependent memory accesses. Some naive approaches to hide such redundant access patterns, such as ORAM [17], would incur prohibitive communication overhead. Therefore, this naturally raises a fundamental research question:

Can we design an MPC protocol that leverages a similar hierarchical graph-based idea to achieve near-logarithmic search complexity?

1.1 Our Results

In this paper, we give a positive answer to the above question by proposing the first MPC ANN protocol with logarithmic online query complexity. Our protocol starts from the SOTA hierarchical graph index HNSW. We first study the single-query setting. To make the search logic fully oblivious, we design a data-independent control flow with a fixed search-termination bound and redesign all data structures as fixed-size arrays that support oblivious updates. To achieve an oblivious access pattern, we combine offline random reshuffling with a duplicated vector database, yielding $O(1)$ online overhead for each graph-index and vector access. Overall, our single-query MPC protocol achieves logarithmic online search complexity, at the cost of at most a 2× increase in vector storage. Moreover, we extend this design to support multiple queries. To handle repeated accesses across queries while avoiding additional online reshuffling, we introduce cross-query "double visited" structures that cache the graph nodes and vectors accessed by earlier queries, enabling later queries to reuse the cached information rather than revisiting the graph index and vector database. In summary, our results are as follows:

Scheme	Communication		Round	
	Single Query	Multiple Queries	Single Query	Multiple Queries
SANNS [11]	$\tilde{O}(N^{1/2}d)$	$\tilde{O}(U\,N^{1/2}d)$	$\tilde{O}(1)$	$\tilde{O}(U)$
PANTHER [27]	$\tilde{O}(N^{1/2}d)$	$\tilde{O}(U\,N^{1/2}d)$	$\tilde{O}(1)$	$\tilde{O}(U)$
HNSecW (Ours)	$\tilde{O}(d)$	$\tilde{O}(Ud+U^2)$	$\tilde{O}(1)$	$\tilde{O}(U)$

Table 1: Online costs for top-k ANN. Communication and round complexity for one query and U sequential queries over N vectors of dimension d, with $k = O(1)$. $\tilde{O}(\cdot)$ hides polylogarithmic factors in N and d.

(1) **Single query.** For a single query, HNSecW achieves $O(\log N)$ online searching complexity over a database of size N. In particular, as shown in Table 1, HNSecW substantially outperforms state-of-the-art secure ANN baselines (SANNS [11], PANTHER [27]), whose clustering-based pipelines have online communication that scales as $\sqrt{N}$.
(2) **Multiple queries.** For U sequential queries, we propose a multi-query protocol with an amortized cost of $O(U \log N)$. Unlike schemes that require reshuffling per query (e.g., PANTHER [27]), our protocol allows U queries to share a single offline reshuffle, avoiding additional online overhead.
(3) **Parallelism and batch queries.** Additionally, we design parallel implementations of the above protocols: for the single-query protocol, the critical-path length is $O(\log N)$, and for the multiple-query protocol, it is $O(U \log N)$. We also propose a batch-query protocol with higher parallelism that processes a batch of U queries simultaneously with a much lower critical-path length of $O(\log^2 U + \log N)$.
(4) **Empirical evaluation.** We conduct a comprehensive experiment on large-scale ANN benchmarks. HNSecW reaches recall over 0.9, matching the state-of-the-art plaintext ANN algorithm. At the same target recall, HNSecW achieves up to 22× WAN speedup and 46× less communication versus the 2PC PANTHER baseline; against the exact-top-k 3PC SORT baseline, it achieves up to 1.6k× WAN speedup and 617× less communication. All code is now available [1].

[1] https://github.com/Kleenex-ultrasoft/Secure_ANN

2 PRELIMINARIES

We introduce notation and background on approximate nearest neighbor search, HNSW algorithm, secure multi-party computation, and oblivious RAM.

2.1 Notation

Let $\mathcal{D} = \{\vec{v}_0, \vec{v}_1, \ldots, \vec{v}_{N-1}\}$ be a dataset of N vectors in $\mathbb{R}^d$, where d is the dimension. We use $\vec{v}[j]$ to denote the j-th coordinate of vector $\vec{v}$. Let $dist : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}_{\geq 0}$ denote a vector distance function. We use $[N] := \{0, 1, \ldots, N-1\}$ to denote the index set. For any set $\mathcal{S}$, $|\mathcal{S}|$ is its cardinality. A graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ has a vertex set $\mathcal{V} \subseteq \mathcal{D}$ and an undirected edge set $\mathcal{E}$. For any vector $\vec{v} \in \mathcal{V}$, we write $\mathcal{N}(\vec{v})$ to denote the set of its neighbors.

2.2 k-Nearest Neighbor Search

For the k-nearest neighbor (k-NN) problem, we have a dataset $\mathcal{D}$ of N vectors in $\mathbb{R}^d$ and a query vector $\vec{q} \in \mathbb{R}^d$. The target is to identify the subset $\mathcal{D}_k^*(\vec{q}) \subseteq \mathcal{D}$ such that $|\mathcal{D}_k^*(\vec{q})| = k$, and for any $\vec{v} \in \mathcal{D}_k^*(\vec{q})$ and any $\vec{v}' \in \mathcal{D} \setminus \mathcal{D}_k^*(\vec{q})$, it holds that $dist(\vec{v}, \vec{q}) \leq dist(\vec{v}', \vec{q})$. A straightforward approach is the linear scan, which computes the distance from $\vec{q}$ to all N vectors and maintains the k smallest distances found so far, typically using a priority queue of size k. The overall computational cost of this method is $O(Nd + N \log k)$. This cost becomes prohibitive when dealing with large-scale datasets (large N) or high-dimensional data (large d).

To address the $O(Nd + N \log k)$ cost, graph-based k-NN approaches have been proposed. These methods operate on a graph $\mathcal{G} = (\mathcal{D}, \mathcal{E})$ over the dataset $\mathcal{D}$, where each vector in $\mathbb{R}^d$ is represented as a vertex, and edges connect pairs of vectors that are sufficiently "close" to each other. Given a query $\vec{q}$, the search procedure greedily traverses the graph from an initial node toward nodes that are closer to $\vec{q}$. A representative example of such a structure is the Delaunay graph [13]. In the two-dimensional case, each node in the Delaunay graph has an expected degree of $O(1)$, and it guarantees that for any query $\vec{q}$, the exact nearest neighbor can be found in expected $O(\log N)$ time [31]. However, moving towards the general case, the average node degree in the Delaunay graph grows exponentially in vector dimension with $O(N^{\lceil d/2 \rceil})$ [13], making search performance impractical.

2.3 Approximate k-NN and Hierarchical Navigable Small World

Given that existing k-NN search algorithms are infeasible for large-scale, high-dimensional data, where both linear scan and ideal graph-based methods are prohibitively expensive, a practical trade-off between accuracy and efficiency is necessary. Consequently, people have proposed the *Approximate k-NN (ANN)*. In ANN, instead of returning the exact nearest set $\mathcal{D}_k^*(\vec{q})$, the goal is to find an approximate set $\mathcal{S}$ that meets a target Recall $R \in (0, 1]$, defined as $\text{Recall}(\mathcal{S}) = |\mathcal{S} \cap \mathcal{D}_k^*(\vec{q})|/k \geq R$. Over the years, various families of ANN methods have been developed, including hashing-based, tree-based, and quantization-based approaches. Among them, Hierarchical Navigable Small World (HNSW) [31], achieves state-of-the-art query performance for high-dimensional data [31].

Algorithm 1 SearchLayer: Greedy search in HNSW layer

Input: Query $\vec{q}$, enter point ep, search width ef, layer ℓ
Output: Set W of ef closest neighbors to $\vec{q}$ in layer ℓ

```
V ← {ep}                                          ▷ Visited set
C ← {ep}                                          ▷ Candidate set
W ← {ep}                                          ▷ Result set
while |C| > 0 do
    c ← extract nearest element from C to q
    f ← furthest element from W to q
    if dist(c, q) > dist(f, q) then
        break                                     ▷ Cannot improve W further
    for each e ∈ N_ℓ(c) do
        if e ∉ V then
            V ← V ∪ {e}
            f ← furthest element from W to q
            if dist(e, q) < dist(f, q) or |W| < ef then
                C ← C ∪ {e}
                W ← W ∪ {e}
                if |W| > ef then
                    remove furthest element from W to q
return W
```

The core of HNSW is a multi-layered graph structure $\mathcal{G} = \{\mathcal{G}_0, \mathcal{G}_1, \ldots, \mathcal{G}_{L-1}\}$, where each layer $\mathcal{G}_\ell = (\mathcal{V}_\ell, \mathcal{E}_\ell)$ denotes a graph whose nodes correspond to vectors in the dataset $\mathcal{D}$. The layers form a sequence of nested subsets of $\mathcal{D}$: $\mathcal{V}_{L-1} \subset \mathcal{V}_{L-2} \subset \cdots \subset \mathcal{V}_1 \subset \mathcal{V}_0 = \mathcal{D}$. The number of nodes decreases geometrically from bottom to top, resulting in $L = O(\log N)$ layers. The edges are constructed based on the underlying vector representations of the nodes, and each $\mathcal{G}_\ell$ serves as an out-degree bound $M = O(1)$ approximation of a Delaunay graph. This hierarchical graph $\mathcal{G}$ acts as an efficient indexing structure for the vector database $\mathcal{D}$. Its design enables a two-phase search strategy: upper, sparser layers support coarse-grained navigation, ensuring high search efficiency, while lower, denser layers provide fine-grained refinement, ensuring high search accuracy.

For query $\vec{q} \in \mathbb{R}^d$, HNSW search begins at a random entry point in the top layer $\mathcal{G}_{L-1}$. Each layer search locates the locally closest node to $\vec{q}$, which becomes the entry point for the next lower layer. After the searching finishes at the base layer $\mathcal{G}_0$, the algorithm further returns the k nearest neighbors of query $\vec{q}$ from all visited nodes. To implement this idea, at each layer the search maintains three data structures: a candidate set C, which stores potential nodes for further exploration; a result set W, which stores the closest nodes found so far, where $|W| = ef$ with $ef = 1$ for non-base layers and $ef = O(k)$ for the base layer; and a visited set V, which records all visited nodes to avoid revisiting them.

All these data structures are initialized with the entry point ep, and the search then proceeds iteratively. At each iteration, we extract the nearest candidate c from C, explore each unvisited neighbor e of c, and add them to V. If e is closer to $\vec{q}$ than the farthest node in W, it is inserted into both C and W, and W is kept at size at most ef. The search terminates when no candidate in C is closer to $\vec{q}$ than the farthest node in W, or when C becomes empty. The detailed procedure is shown in Algorithm 1.

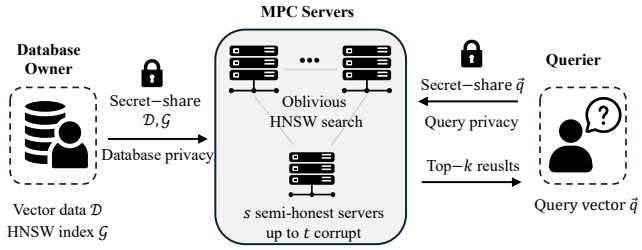


Figure 1: System overview of Oblivious HNSW

Functionality $\mathcal{F}_{\text{HNSW}}[Q]$

Public Input: $Q : (\mathbb{R}^d)^N \times \mathcal{G} \times \mathbb{R}^d \to (\mathbb{R}^d)^k$.
Inputs: $\mathcal{D} = \{\vec{x}_i\}_{i=0}^{N-1} \subset \mathbb{R}^d$, $\vec{q} \in \mathbb{R}^d$, and $\mathcal{G}$
Compute: Upon receiving (dataset, $\mathcal{D}, \mathcal{G}$) from DO, and (query, q) from Querier, where $\mathcal{G}$ is an HNSW graph, compute $(\vec{v}_0, \ldots, \vec{v}_{k-1}) \leftarrow Q(\mathcal{D}, \mathcal{G}, \vec{q})$
Outputs: Upon receiving (continue) from all servers, send $(\vec{v}_0, \ldots, \vec{v}_{k-1})$ to Querier.

Figure 2: Ideal functionality for securely computing HNSW.

We abstract the HNSW algorithm into three key components: the **search logic flow**, the **data structures**, and the **graph and vector access**. The logic flow specifies how the search proceeds within each layer. In each step, it moves to the nearest node in the candidate set and then explores all of its neighbors. The search stops when no candidate is closer to the query than the current results. The data structures support this process. At each layer, the algorithm maintains three sets: a candidate set C, a result set W, and a visited set V. Finally, the logic flow interacts with the underlying graph index and vector database. For each candidate element c, the search first accesses the graph index to retrieve its neighbor list, and then accesses the vector database to obtain the vector of these neighbors for distance computation.

2.4 Secure Multi-Party Computation

Secure multi-party computation (MPC) enables multiple parties to jointly compute a function over their private inputs while only revealing the output to designated parties [47]. We use MPC protocols based on secret sharing where, for each intermediate value x in the computation, all parties hold its secret sharing denoted as $[\![x]\!]$. The security ensures that the adversary cannot learn the value even when corrupting some number of parties as allowed by the threat model. Our system uses MPC protocols as black boxes; thus, we will describe the computation over secret shares directly for clarity. In particular, we use $[\![x]\!] + [\![y]\!]$ and $[\![x]\!] \times [\![y]\!]$ to denote computing the addition and multiplication in MPC over secret-shared values. Typically, given network bandwidth BW and round-trip time RTT, MPC protocols follow the following online latency model:

$$T_{\text{online}} \approx \text{Round} \cdot \text{RTT} + \frac{\text{Communication}}{\text{BW}} + \text{Computation}.$$

Here, Computation is each party's local, non-interactive work and is often almost free. Communication is the total amount of data exchanged, and it scales linearly with the number of multiplication gates. Round is the number of sequential interaction steps, determined by the circuit's multiplicative depth.

Oblivious Access in MPC. MPC protocols based on secret-sharing can typically support any circuit, but random accesses are expensive to represent as circuits. To support private accesses on large arrays without incurring a cost linear to the array, one would need to use an Oblivious RAM (ORAM) [17] by executing the ORAM client in MPC [18]. This type of protocol is also known as Distributed ORAM (DORAM). Given a secret-shared array $[\![\vec{v}]\!]$ and secret-shared index $[\![i]\!]$, DORAM enables parties to jointly compute $[\![\vec{v}[[\![i]\!]]]\!]$ (the secret-shared value at secret-shared position) without revealing the index or the value to any party. DORAM has been optimized for performance in different ways, including circuit-based minimization (e.g., [43]) and protocol-level adaptation (e.g., [14]). Although the best constructions can achieve $O(\log N)$ asymptotic complexity, almost all ORAMs can provide significant savings only when the array size is large.

3 SYSTEM OVERVIEW

We consider a private HNSW search involving two input-holding parties and a set of s MPC servers; Figure 1 gives the high-level architecture. The two input parties are: (1) the database owner (DO), who holds a private vector dataset $\mathcal{D} = \{\vec{x}_i\}_{i=0}^{N-1} \subset \mathbb{R}^d$ together with an HNSW graph index $\mathcal{G}$ built over $\mathcal{D}$, and (2) the Querier, who holds the private query vector $\vec{q} \in \mathbb{R}^d$. DO and Querier are two logical parties who do not need to trust each other, but they could be instantiated as the same physical party, in which case the protocol reduces to the standard outsourced computation model. The functionality of the protocol is shown in Figure 2.

3.1 Threat Model

We assume a semi-honest, probabilistic polynomial-time (PPT) adversary who may corrupt at most t MPC servers and one or both input parties. Our proposed protocol achieves the standard simulation-based security [7]. For any PPT adversary corrupting at most t MPC servers and one or both input parties, there exists a PPT simulator whose output is computationally indistinguishable from the adversary's output, using only the corrupted parties' inputs, their outputs under $\mathcal{F}_{\text{HNSW}}$, and the public parameters. We provide the full proofs in Appendix A. In particular, public parameters are summarized in Table 2.

Category	Public Parameters
Data characteristics	Vector dimension d; Dataset size N; Number of layers L; Degree bound M; Per-layer sizes $\{N_\ell\}_{\ell=0}^{L-1}$
Search hyperparameters	Result set size ef; Nearest neighbor output size k
Multi/Batch-query setting	Number of queries U

Table 2: Public parameters of the system.

The ideal functionality and the threat model imply symmetric privacy and correctness. For **query privacy**, the database owner

and all MPC servers learn nothing about $\vec{q}$ beyond what is implied by the public parameters. For **database privacy**, the querier and all MPC servers learn nothing about $\mathcal{D}$ or $\mathcal{G}$ beyond the top-k result vectors output by $\mathcal{F}_{\text{HNSW}}$ and what is implied by the public parameters. For **correctness**, for any inputs $(\mathcal{D}, \mathcal{G}, \vec{q})$, the output reconstructed by the Querier equals the output of the ideal functionality $\mathcal{F}_{\text{HNSW}}$.

3.2 Single-Query

Even when all vectors and their distance to the query are kept secret, a single HNSW query is not oblivious in its control flow, data structures, or graph and vector access patterns, all of which depend on the underlying graph and the query. Achieving obliviousness to prevent leakage requires a dedicated analysis of the performance features that HNSW presents, since a generic method that follows only worst-case conditions would introduce prohibitive overhead. In our protocol, we design customized optimizations to make each part oblivious by taking advantage of its unique features. For a single query, we traverse each layer without revealing the access pattern. At every layer, we use our oblivious data structure to maintain the search state, and we terminate the traversal using a tailored termination condition that prevents information leakage. We briefly discuss the key ideas here and leave full details in Section 4.

- **Control flow.** We eliminate data-dependent stopping conditions by replacing them with a fixed iteration bound derived from a detailed analysis of HNSW's stopping behavior, ensuring obliviousness with only negligible recall loss (Section 4.1). Although there is no theoretical bound on the iterations, we found that in the specific context of RAG, we could set a tight upper bound on the number of iterations without degrading the end-to-end performance.
- **Data structure.** We observe that both the candidate set and the result set exhibit a monotonic exclusion property: once a node is excluded, it will not be included in these sets again. By using this monotonicity property, we design an oblivious data structure for both sets that keeps them at a fixed size and removes the farther nodes through a fixed-size tail truncation after an oblivious sorting step (Section 4.2).
- **Graph and vector access pattern.** We utilize the fact that HNSW naturally avoids revisiting the same node more than once per layer. To further reduce redundant access across the layers, we assign each layer an independently shuffled graph and vector tables to ensure that every address is accessed at most once, yielding fully oblivious access patterns.

3.3 Multiple-Query

The extension from the single-query protocol to the multi-query setting introduces a fundamentally new challenge: different queries may revisit the same graph node and its associated vector. To hide these repeated visits, one would need to reshuffle the graph and the vector tables after every query. However, such per-query reshuffling is impractical in an online setting due to the significant latency it entails. To address this, we rely on the insight that once data has been accessed, it can be cached and reused across queries rather than accessed repeatedly. We propose to use two global cross-query visited structures, one for graph neighbor lists and one for vector contents. These structures serve as a secure shared cache: once a node or vector is retrieved by any query, it is stored in the global visited array. Subsequent queries check this array first and reuse the cached data. As a result, each graph or vector access proceeds by first scanning the cache and then performing a real or dummy data-access step. Under this design, every node and vector is accessed at most once from the underlying shuffled tables globally, while subsequent revisits are served through oblivious cache scans. Combined with one-time random pre-shuffling, this ensures that the overall access pattern remains oblivious; see Section 5.

3.4 Parallel-Enhanced Batch Model

Furthermore, we consider how the oblivious HNSW search can be executed in parallel. The single-query protocol already exhibits high parallelism, but its multiple-query extension is designed to process the U queries sequentially, giving a linear circuit depth of $O(U)$. In a batch setting where many queries are available simultaneously, it is natural to ask whether the computation can be reorganized to reduce this depth to better exploit parallel execution. As in the multiple-query setting, the fundamental challenge is to avoid accesses to the same nodes and vectors across queries. A naive approach is to cache every accessed element (neighbor lists and vectors), as in the sequential protocol. But in the batch setting, such a large cache is unnecessary. Our key insight is that, because all queries proceed in synchronized iterations, we do not passively wait for cache hits and reuse; instead, we force $U \cdot M$ accessed vectors to be evaluated against U queries: if a query later needs that vector, it can skip the inner path; if not, the extra evaluation does not affect correctness. This eliminates the need for a massive cache and naturally prevents redundant accesses. As a result, the batch-query protocol achieves a depth of $O(\log^2 U + \log N)$, coming from the oblivious duplicate removal of the $U \cdot M$ neighbors per iteration plus the $O(\log N)$ HNSW layers.

3.5 Implementation & Experiment Summary

We implement HNSecW in two MPC configurations: 2PC in ABY [12] and 3PC in MP-SPDZ [24]. We evaluate on 10 ANN benchmarks at M=128 (SIFT-1M, GIST-1M, DEEP-100M, BIGANN-1B, DEEP-1M, DEEP-image, MNIST-60K, Fashion-60K, SPACEV-1B, SSNPP-1B) [4, 22, 25, 35, 44], spanning N from $60K$ to $1B$ and $d \in [96, 960]$. Experiments are conducted under LAN (4000 Mbps, 1 ms RTT) and WAN (320 Mbps, 50 ms RTT). Overall, on accuracy, HNSecW delivers Recall@$k \geq 0.9$ for at least 95% of queries across the full evaluated range of $k \in \{10, 20, 50, 100\}$. On performance, HNSecW outperforms existing work on WAN 2PC by up to 22× in end-to-end latency and 46× in communication.

4 SINGLE QUERY PROTOCOL

As discussed in Section 2, we decompose HNSW search into three components: the search logic flow, the data structures, and the graph/vector access patterns. In this section, we briefly outline how to make each component oblivious. Section 4.1 makes the control flow data-independent by replacing the original data-dependent stopping conditions with a fixed iteration bound. Section 4.2 redesigns the visited, candidate, and result structures as fixed-size arrays with uniform update rules. Section 4.3 ensures oblivious

graph and vector access by avoiding repeated retrievals through per-layer reshuffled tables. Finally, Section 4.4 assembles these building blocks into the complete oblivious single-query protocol.

4.1 Making Logic Flow Oblivious

We first focus on making the HNSW search logic flow oblivious. In the original HNSW algorithm, at each layer, the search algorithm iteratively finds the nearest node from the candidate set and explores all its neighbors. The logic flow of this process is inherently data-dependent.

The first source of data dependence comes from the variation in node degrees: exposing this difference would leak information about the graph and the query. Fortunately, this dependency is easy to remove. Since HNSW enforces an out-degree upper bound M per node, we pad each node's neighbor list with randomly selected nodes from the same layer to obtain a fixed length of M. This padding does not reduce search quality: all real neighbors remain available for selection.

The more subtle challenge lies in the termination condition. In HNSW, the search stops when no better candidate remains (line 7-8 in Algorithm 1), or when the candidate set becomes empty (line 4 in Algorithm 1). Both conditions are inherently data-dependent. At first glance, this seems similar to the bounded-degree case, since the original HNSW paper [31] suggests that each layer can be traversed in a constant number of hops. One may treat that constant as a fixed upper bound and simply pad the execution with dummy searches. However, unlike node degree, this constant is not a worst-case bound; it arises from an expected-case analysis in [31] that assumes each layer forms a perfect Delaunay graph [13]. Real HNSW layers only approximate such structures, so the constant-hop guarantee does not hold in general. In fact, the best-known asymptotic bound for HNSW on arbitrary data is still $O(N)$. A practical, data-independent iteration bound must therefore be derived from the actual termination behavior.

The first termination condition (lines 7–8 in Algorithm 1) triggers when the closest candidate can no longer improve the current result set. This can only happen after ef search steps. Since every node inserted into W is also inserted into C, every node in the final W must have been popped from C and explored (i.e., cannot remain in C); otherwise, it would stay in C and lead to $\min(C) \leq \max(W)$, contradicting the stopping condition. Hence, letting s denote the number of search steps performed by standard HNSW at a given layer, we always have $s \geq ef$ and can write exact search steps as $s = ef + \Delta$, where the residual Δ captures the additional work beyond the ef steps. Therefore, to make the termination condition oblivious, a natural approach is to fix the number of search steps at each layer. Specifically, for each layer ℓ, we set the number of search steps to $T^{(\ell)} = ef^{(\ell)} + \overline{\Delta}^{(\ell)}$, where $\overline{\Delta}^{(\ell)}$ is a public upper parameter to bound the residual term and depends only on public quantities such as N, d, and M. Below, we present our strategy for setting $ef^{(\ell)}$ and $\overline{\Delta}^{(\ell)}$, both of which are bounded by logarithmic terms, and show that the resulting procedure performs robustly in practice. However, designing termination strategies for HNSW that balance accuracy and efficiency remains an active research topic even in the non-private setting [8], and is not the main interest of this paper. Future termination strategies can also be integrated into our framework. Moreover, we highlight that, as long as the number of search steps increases by at most a logarithmic factor, our final complexity guarantee remains unchanged.

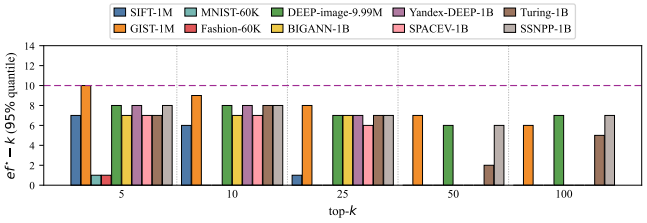


Figure 3: Base-layer ef^* on the 10 datasets (M=128) with 95% recall $\geq$ 0.9 over 10^3 queries.

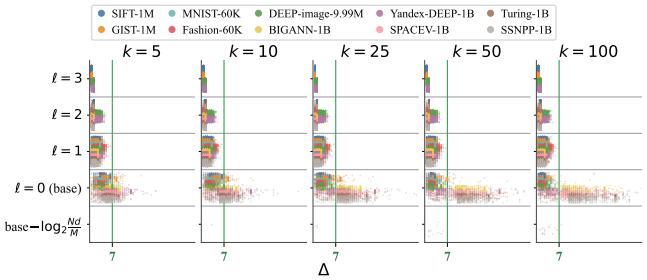


Figure 4: Per-layer residual Δ at M=128, $ef^{(\ell \geq 1)}$=1, $ef^{(0)}$=k+10, on the 10 datasets. The bottom row of each panel shows the base-layer step count minus $\lceil \log_2(Nd/M) \rceil$.

We begin with the choice of $ef^{(\ell)}$. Standard HNSW [31] sets $ef^{(\ell)} = 1$ in all upper (non-base) layers, while treating the base-layer value as an input parameter. We now discuss how to set $ef^{(0)}$ at the base layer. To this end, we run HNSW with five different values of k on 10 ANN datasets spanning 60K–1B points and dimensions 96–960, using 10^3 queries per dataset. For each test, let ef^* denote the smallest base-layer value of $ef^{(0)}$ that achieves recall at least 90%. Figure 3 reports the 95% quantile of ef^* over these 50K tests. For all $k \geq 5$, the required offset $ef^* - k$ is at most 10 in at least 95% of the tests. Therefore, we set $ef^{(0)} = k + 10$ throughout our experiments, except in the ablation study on ef.

After fixing the strategy for $ef^{(\ell)}$, we next discuss the choice of $\overline{\Delta}^{(\ell)}$. One natural idea is to set $\overline{\Delta}^{(\ell)}$ to a single empirical constant shared across all layers. We validate this strategy using the same 50K experiments, where we measure the number of residual steps required by the standard HNSW algorithm when adopting the above strategy for $ef^{(\ell)}$. The results are shown in Figure 4.

The results show that the residual steps are well bounded by a constant in the upper layers, but are substantially larger at the base layer. We therefore use an enhanced strategy. For upper layers, we set $\overline{\Delta}^{(\ell)}$ to a constant C. For the base layer, we use the observation from recent work on early termination in HNSW [8, 9] that, after the first ef steps, the probability of not having terminated decays geometrically with the number of additional steps. We therefore add a logarithmic term in N, d, and M at the base layer: $\overline{\Delta}^{(0)} =$

$\lceil \log_2(Nd/M) \rceil + C$. In our experiments, we set $C = 7$, which covers more than 95% of cases across the 50K tests.

Based on the above discussions, we apply the following fixed-step rule, where we search $1 + 7 = 8$ steps at each upper layer and search $\lceil \log_2(N \cdot d/M) \rceil + ef^{(0)} + 7$ steps at the base layer. Since $\overline{\Delta}^{(\ell)}$ may be larger than the actual number of residual steps, and since the second termination condition in line 4 of Algorithm 1 may trigger early termination, the search may otherwise stop before completing $\overline{\Delta}^{(\ell)}$ steps. In this case, we continue the search using dummy nodes. When dummy nodes are treated as candidates during the search, they are assigned sentinel distances that rank worse than all real candidates, and therefore never affect the result set W. Overall, our strategy yields an oblivious search procedure. Here, the number of search steps is $O(1)$ at each upper layer and $O\big(\log(N \cdot d/M)\big)$ at the base layer. Since HNSW has $O(\log N)$ layers, this gives a total of $O\big(\log(N \cdot d/M) + \log N\big)$ search steps per query.

4.2 Making Data Structures Oblivious

We next make the data structures of HNSW oblivious. Recall that each layer individually maintains three structures: the visited set V, the candidate set C, and the result set W. However, the sizes and update frequencies of the three sets are inherently data- and query-dependent during the search process. We therefore redesign them into oblivious counterparts.

4.2.1 Oblivious Result Set W. The result set W is more straightforward to make oblivious than C and V because its size is already bounded by ef in the original HNSW algorithm. The only data-dependent behavior is the conditional insertion rule, i.e., inserting a node into W when it improves the current ef results and passes the visited check (line 13 in Algorithm 1). We remove this data-dependent condition by obliviously considering every node for insertion, at a cost of $O(ef)$ per node.[2] Overall, this naive approach incurs a cost of $O(M \cdot ef)$ in each iteration. To improve practical efficiency, we instead insert all M elements, apply oblivious sorting, and then reduce the size back to ef, incurring only $O(ef \log^2(ef))$ complexity. Since $\log^2(ef)$ is much smaller than M in our setting, this approach achieves better practical performance than the naive alternative.

4.2.2 Oblivious Visited Set V. The visited set V records explored nodes to prevent revisiting them in the later steps. In the original HNSW, V is a dynamic set whose size and insertion pattern are both query- and data-dependent, since nodes are inserted only when first encountered and their growth is unpredictable. To eliminate this dependence, we observe that V does not require uniqueness and can safely contain duplicate nodes. We therefore replace the set, whose size is unpredictable, with a list whose size increases by exactly one after each exploration. More precisely, in each iteration, we append all M neighbors, regardless of whether they have appeared before, so that V contains exactly $M \cdot T + 1$ entries after T iterations. A visited check is performed by a full linear scan over this list, with a cost of $O(MT)$, which is scalable and independent of N.

[2]One could reduce this to $O(\text{polylog}(ef))$ using more sophisticated oblivious algorithms [1], but these become advantageous only when ef is very large ($\geq 10^4$), which is far beyond our setting.

4.2.3 Oblivious Candidate Set C. The candidate set C stores the nodes awaiting expansion and is also a fully dynamic set. In the original HNSW, its size is inherently data-dependent: each iteration extracts one node from the set but may insert zero, a few, or up to M new nodes. The insertion condition is also data-dependent, since neighbors are added to C only when they pass the visited check and could improve the result set W (line 10, 13 in Algorithm 1). To make C oblivious, we leverage the fixed iteration T established in Section 4.1. Since at most T candidates will be extracted, any entry ranked beyond the top-T positions can never be used and therefore does not need to be retained. We therefore replace the dynamic set with a fixed-length array of size T. In every iteration, all M neighbors, whether real or dummy after the visited check, are appended to C. We then perform an oblivious sort by distance and retain only the top-T entries. As we discussed earlier, since dummy nodes carry a maximum distance $d_{\max}$, they always expand to other dummy nodes. They cannot influence the result set W even if they are selected after the real nodes are exhausted in C. This fixed-size and fixed-update design of C eliminates data-dependent behavior while maintaining the same functionality as the original HNSW.

4.3 Making Access Patterns Oblivious

The HNSW search process involves two types of data access: vector access and graph access. During the search, a node first incurs a vector access where its distance to the query is evaluated. If the node is later selected for expansion (extracted from the candidate list C), it then incurs a graph access to retrieve its neighbor list. A straightforward way to make both accesses oblivious is to store the graphs and the vector database in DORAM. However, this approach introduces significant overhead, as each access requires $O(\log N_\ell)$ time.

To design a more efficient solution, we recall that achieving oblivious access requires two properties: data anonymization, which can be easily achieved through pre-shuffling, and non-repetitive access, where no address is revisited within a query. In the single-query setting considered here, we assume that both the graph and vector data have been anonymized through offline reshuffling. Therefore, if each item is retrieved at most once, the access is oblivious. We now analyze where repeated accesses arise, and how they can be eliminated.

Within a single layer, both graph and vector accesses are non-repetitive. The visited set prevents revisiting any node, so its neighbor list and its vector are retrieved at most once. Across layers, the search may encounter the same logical node again, because each layer builds a separate, increasingly refined neighborhood structure, and the same node provides different information at each layer. So its data may be reused along the search path. For graph access, this does not cause any address repetition: each layer stores its own graph with independently reshuffled addresses, so expanding the same logical node in different layers always reads unrelated locations. For vector access, however, standard HNSW implementations keep all vectors in a single global table for space and cache efficiency, so re-evaluating a node at multiple layers would hit the same physical address of the vector table. To avoid this, we assign each layer its own independently reshuffled vector table of size N_ℓ. Notably, within a single layer, the graph and vector tables are

reshuffled independently, so a node's graph entry and its vector entry occupy unrelated physical addresses, avoiding graph-to-vector transitions by reusing an address. With this per-layer duplication and independent reshuffling, both graph and vector access become fully non-repetitive.

4.4 Put It All Together

Having established the oblivious control flow, data structures, and access patterns, we now integrate these components into a complete single query HNSW protocol under MPC. Here, we first describe how the per–layer data is represented and secret–shared. At each layer ℓ, the data owner provides two tables: a graph table $\mathcal{G}_\ell$ containing neighbor lists as the HNSW index and a vector table $\mathcal{D}_\ell$ containing the node vectors. Both tables are padded with $TM{+}T{+}ef$ dummies, where TM is for visited check, while $T + ef$ is for initialization of C and W. After padding, the tables are secret–shared and jointly reshuffled by MPC parties in an offline procedure. Each graph table entry stores M neighbors' vector-table addresses, $\mathcal{G}_\ell[i] = (\llbracket n_{i,0} \rrbracket, \ldots, \llbracket n_{i,M-1} \rrbracket)$, where $\mathcal{D}_\ell[n_{i,j}]$ would return the j-th neighbor's vector and auxiliary information. Each vector table entry is defined as $\mathcal{D}_\ell[j] = (\llbracket \vec{v}_j \rrbracket, \llbracket a_j \rrbracket, \llbracket b_j \rrbracket, \llbracket m_j \rrbracket)$, where $\vec{v}_j$ is the node's vector; a_j is the graph table address of the nodes; b_j is the address of the same vector in $\mathcal{D}_{\ell-1}$ (used during layer transition); and $m_j \in \{0, 1\}$ is the dummy entry flag. We collect all dummy vector addresses in a pool $\mathrm{DUM}_\ell = (\llbracket d_0 \rrbracket, \ldots, \llbracket d_{MT+T+ef-1} \rrbracket)$.

During the search, MPC parties jointly maintain three fixed-size arrays at each layer: $V = \{\llbracket a_i \rrbracket\}_{i=0}^{MT}$, $C = ((\llbracket c_i \rrbracket, \llbracket d_i \rrbracket))_{i=0}^{T-1}$, and $W = ((\llbracket w_i \rrbracket, \llbracket b_i \rrbracket, \llbracket \delta_i \rrbracket))_{i=0}^{ef-1}$. Here V stores the vector-table addresses a_i of visited nodes; C stores the graph-table addresses c_i and distances d_i of candidate nodes; W stores the vector-table address w_i, the lower-layer vector address b_i, and the distance δ_i of the current top-ef results nodes.

With the oblivious data structures in place, Algorithm 2 specifies the per-layer search, which runs a fixed number of iterations and updates V, C, and W in an oblivious manner. The full single-query protocol (Algorithm 3) applies this layer search from the top layer down to the base. At the base layer, W_0 stores only vector-table addresses to keep the oblivious sort efficient; the actual vectors are fetched from an auxiliary reshuffled table $\mathcal{D}_{-1}$. Considering the base layer may place dummy addresses among the top-k when k is large, $\mathcal{D}_{-1}$ is padded with $MT{+}T{+}ef$ dummy entries containing application-safe vectors so that dummy outputs remain indistinguishable from real ones.

We analyze the online complexity of the single-query protocol. At the base layer, we set $ef = O(k)$ and run $T = O(k)$ iterations. Each iteration performs: $M = O(1)$ distance evaluations in $\mathbb{R}^d$; M oblivious scans over a list V of size $O(k)$; the oblivious sort network evaluation on C and W, each of size $O(k)$. Over T iterations, both communication and local computation are $O\left(kd + k^2 \log^2 k\right)$. For $O(\log N)$ non-base layers, with $ef = 1$, the cost becomes $O(d \log N)$. Therefore, a single oblivious HNSW query has communication and local computation costs $O(kd{+}k^2 \log^2 k{+}d \log N)$. The single-query security proof is provided in Appendix A.4.

Algorithm 2 ObliviousSearchLayer: Oblivious layer search

Input: Query $\vec{q}$, entry point ep, search width ef, slack $\overline{\Delta}^{(\ell)}$, degree bound M, layer ℓ (per-layer tables $G_\ell, D_\ell, \mathrm{DUM}_\ell$ accessed implicitly)
Output: Set W of ef closest neighbors to $\vec{q}$ in layer ℓ

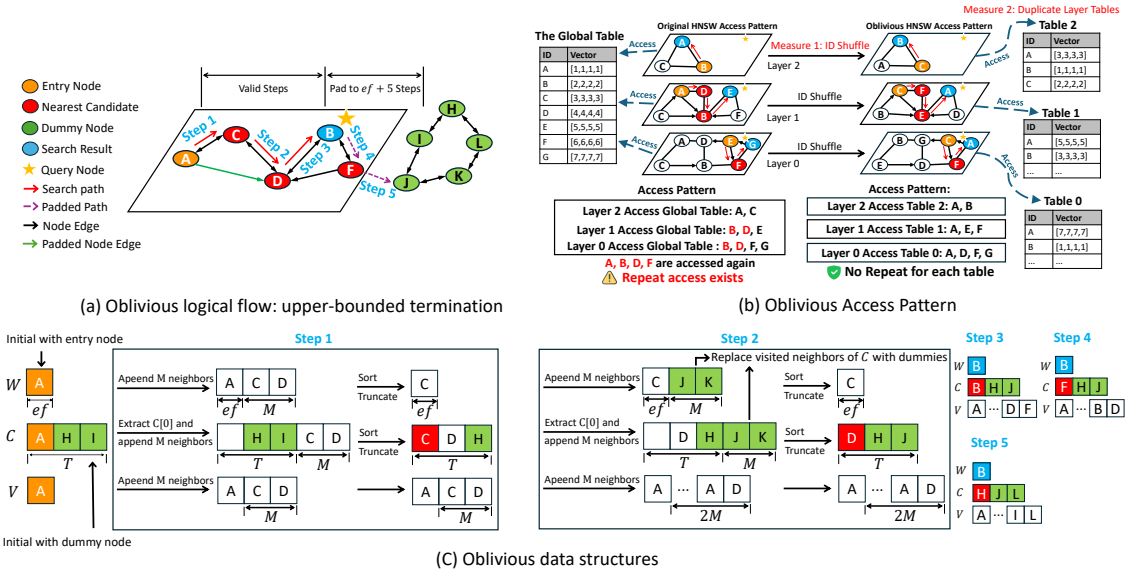
$T \leftarrow ef + \overline{\Delta}^{(\ell)}$
initialize visited array $V[0 : T \cdot M] \leftarrow \sigma_{\text{addr}}$ ▷ dummy graph address
initialize candidate array $C[0 : T + M - 1] \leftarrow (\sigma_{\text{addr}}, d_{\max})$
initialize result array $W[0 : ef + M - 1] \leftarrow (\sigma_{\text{addr}}, \sigma_{\text{addr}}, d_{\max})$
$(\vec{v}_{ep}, a_{ep}, b_{ep}, m_{ep}) \leftarrow D_\ell[ep]$
$d_{ep} \leftarrow dist(\vec{v}_{ep}, \vec{q})$
$V[0] \leftarrow a_{ep}$
$C[0] \leftarrow (a_{ep}, d_{ep})$
$W[0] \leftarrow (ep, b_{ep}, d_{ep})$
for $t = 0$ **to** $T - 1$ **do**
  $(c, d_c) \leftarrow C[0]$ ▷ current best candidate
  shift $C[1 : T - 1]$ left by one position
  $C[T - 1] \leftarrow (\sigma_{\text{addr}}, d_{\max})$
  $(n_0, \ldots, n_{M-1}) \leftarrow G_\ell[c]$ ▷ neighbors of c in layer ℓ
  for $i = 0$ **to** $M - 1$ **do**
    $u \leftarrow n_i$
    $visited \leftarrow \text{LinearScan}(V, u)$
    $dummy \leftarrow \mathrm{DUM}_\ell[T + ef + t \cdot M + i]$
    $u' \leftarrow \text{Select}(visited, dummy, u)$
    $(\vec{v}_u, a_u, b_u, m_u) \leftarrow D_\ell[u']$
    $mask \leftarrow visited \vee m_u$
    $d_u \leftarrow \text{Select}(mask, d_{\max}, dist(\vec{v}_u, \vec{q}))$
    $V[t \cdot M + i + 1] \leftarrow a_u$
    $C[T + i] \leftarrow (a_u, d_u)$
    $W[ef + i] \leftarrow (u', b_u, d_u)$
  $C \leftarrow \text{ObliviousSortTruncate}(C, T)$ ▷ sort $(T{+}M)$ and keep best T
  $W \leftarrow \text{ObliviousSortTruncate}(W, ef)$ ▷ sort $(ef{+}M)$ and keep best ef
return the first ef entries of W

5 MULTIPLE QUERY PROTOCOL

In this section, we extend the single-query protocol to support U queries. Recall that the single-query protocol performs an oblivious search over tables that are pre-shuffled in an offline phase. A naive approach for U queries is to run the single-query protocol U times, reshuffling the tables after each query, but this incurs prohibitive online costs due to the $(U - 1)$ additional reshuffles. Our objective is therefore to answer all U queries using a single shuffled HNSW index with no further online reshuffling. To achieve this, we propose using two global cross-query visited structures, one for graph nodes and the other for vectors, so that previously accessed information can be safely reused in later queries without re-accessing the underlying tables.

5.1 Double Visitor Arrays

The challenge in the multiple-query setting is that later queries may need to access nodes and vectors that earlier queries already



(a) Oblivious logical flow: upper-bounded termination

(b) Oblivious Access Pattern

(C) Oblivious data structures

Figure 5: Overview of Oblivious HNSW Components. (a) Each layer executes exactly $ef + \Delta$ steps, ensuring data-independent termination. (b) Per-layer replicated and reshuffled tables hide cross-layer access patterns. (c) Oblivious data structures maintain the HNSW search state.

Algorithm 3 OBLIVIOUSSINGLEQUERY: HNSW single query

Input: Query $\vec{q}$; top-layer entry $i^{\mathcal{D}}_{\text{top}}$; tables $\{\mathcal{G}_\ell, \mathcal{D}_\ell\}_{\ell=0}^{L-1}$ and $\mathcal{D}_{-1}$; params $(ef, \overline{\Delta}, M)$; result count k.

Output: Top-k nearest neighbor vectors of $\vec{q}$.

$i^{\mathcal{D}}_{\text{ep}} \leftarrow i^{\mathcal{D}}_{\text{top}}$
for $\ell = L-1$ **to** 1 **do**
  $W_\ell \leftarrow$ OBLIVIOUSSEARCHLAYER$(\vec{q}, i^{\mathcal{D}}_{\text{ep}}, ef = 1, \overline{\Delta}^{(\ell)}, M, \ell)$
  $(w^{\mathcal{D}}, b^{\mathcal{D}}, \delta) \leftarrow W_\ell[0]$
  $i^{\mathcal{D}}_{\text{ep}} \leftarrow b^{\mathcal{D}}$
$W_0 \leftarrow$ OBLIVIOUSSEARCHLAYER$(\vec{q}, i^{\mathcal{D}}_{\text{ep}}, ef, \overline{\Delta}^{(0)}, M, \ell = 0)$
Reveal $\{b^{\mathcal{D}}_i\}_{i=0}^{k-1}$ in W_0 ▷ top-k vector addresses in additional table
Fetch final top-k vectors $\{\vec{v}_i\}_{i=0}^{k-1} = \{\mathcal{D}_{-1}[b^{\mathcal{D}}_i]\}_{i=0}^{k-1}$
Reconstruct $\{\vec{v}_i\}_{i=0}^{k-1}$ to Querier
return $\{\vec{v}_i\}_{i=0}^{k-1}$

---

accessed. As mentioned before, the single-query protocol prevents repeated data access by using an array V to indicate which nodes we have already accessed. Moving to the multiple-query setting, one straightforward idea is to simply make V global, so that each node can be accessed by only one query. However, multiple queries may overlap in their search paths, so simply globalizing V would break the correctness of multi-query HNSW search.

To address this issue, we propose two global cross-query cache structures that record all nodes and vectors accessed so far. Specifically, we maintain a graph-side visited set $V^{\mathcal{G}}_\ell$ and a vector-side visited set $V^{\mathcal{D}}_\ell$, which store the neighbor information of visited nodes and the contents of visited vectors, respectively. When exploring a node v in graph $\mathcal{G}_\ell$, we first check whether $v \in V^{\mathcal{G}}_\ell$. If so, the neighbor information of v is retrieved from $V^{\mathcal{G}}_\ell$, followed by an access to a dummy node in the graph. Otherwise, v, together with all of its neighbor information, is read from $\mathcal{G}_\ell$ and then stored in $V^{\mathcal{G}}_\ell$ for future use. After obtaining the neighbor information, we compute the distance between the query and each neighbor, which requires accessing the vector associated with each neighbor. Similarly, for each vector access, we first check $V^{\mathcal{D}}_\ell$. If the vector is already present, it is retrieved from $V^{\mathcal{D}}_\ell$, followed by a dummy vector access. Otherwise, it is loaded from $\mathcal{D}_\ell$ and appended to $V^{\mathcal{D}}_\ell$. Moreover, before processing multiple queries, we pre-shuffle both the graphs and the vector databases so that all nodes and vectors, including dummy ones, remain anonymous.

More precisely, for each graph access, we first perform a linear scan of $V^{\mathcal{G}}_\ell$. Then, depending on whether the target node is already in the cache, we access either the real node or a dummy node in the graph. Finally, we access $V^{\mathcal{G}}_\ell$ again to store either the newly retrieved real node or a dummy entry. Notably, we generate $U \times T$ dummy nodes so that for each graph access, we can access the different dummy nodes. With this design, the control flow is identical for every graph access. Regarding the access pattern, all nodes, including dummy nodes, are accessed at most once, thereby guaranteeing non-repetitive access. In addition, the graph is pre-shuffled to ensure data anonymization. Similarly, for each vector access, we first scan $V^{\mathcal{D}}_\ell$, then access either a real or dummy vector, and finally store it in $V^{\mathcal{D}}_\ell$. By the same argument as for graph access, these vector accesses are also non-repetitive and data-anonymized, and hence oblivious. Together, the graph and vector access procedures ensure that the overall multi-query access pattern remains oblivious.

We analyze the online complexity of the multiple-query protocol for U queries. At the base layer, we keep $ef = O(k)$ and run $T =$

$O(k)$ iterations per query. Each iteration performs: $M = O(1)$ distance evaluations in $\mathbb{R}^d$; M oblivious scans over $V^{\mathcal{D}}$ and $V^{\mathcal{G}}$ of size $O(Uk)$; and two oblivious sort network evaluations over C and W, each of size $O(k)$. Thus, both communication and local computation costs at the base layer are $O(Ukd + Uk^2 \log^2 k + U^2k^2)$, and for the $O(\log N)$ non-base layers with $ef = 1$ they are $O((Ud + U^2) \log N)$. Therefore, the multiple-query protocol has total communication and local computation costs $O\big(Uk(d + k \log^2 k + Uk) + (Ud + U^2) \log N\big)$. The multi-query security proof is provided in Appendix A.5.

6 BATCH QUERY PROTOCOL IN PARALLEL

Previously, we presented the multiple-query protocol for the sequential setting, where queries are processed one after another. We further consider the batch setting, where U queries arrive together. We propose a batch protocol that processes the U queries in parallel, reducing the round complexity from $O(U)$ to $O(\log^2 U)$.

The key idea is to share vectors fetched by any query across all U queries, so every query steps forward together by updating its own C and W. Each iteration has two stages. First, the protocol obliviously deduplicates the nearest candidate from each query's C, replaces duplicates with dummies, and issues U distinct graph accesses. Second, it obliviously deduplicates the $U \cdot M$ returned neighbor IDs and fetches $U \cdot M$ vectors. Each query evaluates its distance to all $U \cdot M$ fetched vectors, regardless of which query requested each vector. The batch therefore performs $U^2 \cdot M$ distance evaluations in parallel. These distances update the query's own candidate set C and result set W. The protocol records every fetched vector in a single global visited array V shared by all queries. Each of the two oblivious deduplication steps sorts $O(U)$ elements, requiring an oblivious sort network of depth $O(\log^2 U)$, which determines the per-iteration round cost. The full algorithm and the round-complexity proof appear in Appendix B and Appendix A.6.

7 EVALUATION

In this section, we empirically evaluate HNSecW and aim to answer the following questions:

- **Search quality.** What search quality does HNSecW achieve, and how does it perform when integrated into downstream RAG?
- **Overhead.** What computation and communication overhead does HNSecW incur, and how does its performance compare against state-of-the-art secure ANN constructions?

7.1 Experimental Setup

Dataset	N	d	Metric	Dataset	N	d	Metric
SIFT-1M	10^6	128	ℓ_2	DEEP-image	10^7	96	Cos.
DEEP-1M	10^6	96	ℓ_2	DEEP-100M	10^8	96	ℓ_2
GIST-1M	10^6	960	ℓ_2	BIGANN-1B	10^9	128	ℓ_2
MNIST-60K	6×10^4	784	ℓ_2	SPACEV-1B	10^9	100	ℓ_2
Fashion-60K	6×10^4	784	ℓ_2	SSNPP-1B	10^9	256	ℓ_2

Table 3: ANN datasets used in our evaluation [3, 35].

MPC Backends. HNSecW can be instantiated with any semi-honest MPC protocol that tolerates t corruptions out of s servers by using its corresponding MPC primitives. Following commonly used setups of existing studies, we use the ABY [12] and MP-SPDZ framework [24] in two distinct security configurations for apples-to-apples comparison with existing secure ANN/KNN baselines:

- Semi-honest 2PC ($s = 2, t = 1$). Two computation servers jointly run the HNSW search, with at most one corrupted server. We implement 2PC HNSecW in ABY [12].
- Semi-honest 3PC ($s = 3, t = 1$). Three computation servers with at most one corrupted server. We implement 3PC HNSecW in MP-SPDZ [24].

Hardware and Network Environment. All experiments are conducted on a single server with an AMD EPYC 9554P CPU (64 physical cores, 3.1 GHz) and 565 GB of RAM, where each MPC party runs as a separate single-threaded process. We emulate two network conditions: a LAN with 4000 Mbps bandwidth and 1 ms RTT (a standard MPC LAN setting [11, 27]), and a WAN with 320 Mbps bandwidth and 50 ms RTT, following typical U.S. East–West latency per cloudping.co[3].

Datasets. We evaluate on 10 widely-used ANN benchmarks [3, 35] (Table 3), spanning $N \in [6\times10^4, 10^9]$, $d \in [96, 960]$, and both ℓ_2 and angular metrics. The selected scale range covers the practical RAG deployment regime. Due to space limitations, the main text reports four representative datasets covering both scales (SIFT-1M, GIST-1M, BIGANN-1B, DEEP-100M); results for the remaining six datasets are provided in Appendix C. All recall experiments use 10^3 queries per dataset and report the 95th-percentile statistic.

Comparison Baselines. Since HNSecW targets symmetric privacy, both the database and the query are secret-shared across non-colluding MPC servers. Existing secure ANN protocols mostly target the asymmetric setting, a less general special case of ours: they either keep the database public [49] or have one party hold it in plaintext [11, 27], so a direct comparison would unfairly favor those baselines. For an apples-to-apples comparison we therefore generalize the closest prior protocols to symmetric privacy by preserving their algorithms unmodified and only replacing the underlying primitives. For the 2PC baseline, Tiptoe [20] is an early secure ANN protocol where the query is private and one party holds the plaintext database. SANNS [11] and PANTHER [27] follow the same clustering-based pipeline and are closest to our setting. Both can be generalized to symmetric privacy by storing the database in secret-shared form; since PANTHER further improves SANNS in end-to-end performance, we use it as the representative baseline. Specifically, we generalize PANTHER by replacing its plaintext-database PIR primitive with DORAM [38] and evaluating distance computation on secret-shared inputs. The PANTHER algorithm itself (cluster construction, top-k circuit, candidate aggregation) is unchanged; for full transparency, we report the original cost and the generalization overhead as separate components in Figure 9. For the 3PC baseline, we compare against exact top-k using the replicated radix sort of Asharov et al. [1], the most efficient secure sorting protocol.

7.2 Search Accuracy and RAG Quality

Search Recall and Distribution. At the deployed setting (M=128, $ef^{(0)}$=k+10, with $\overline{\Delta}^{(\ell)}$ from Section 4.1), HNSecW delivers recall ≥ 0.9 for at least 95% of queries on every dataset and k, with the

[3]https://www.cloudping.co/

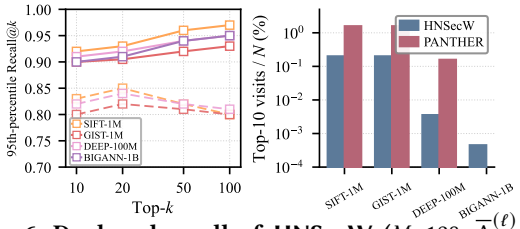


Figure 6: Deployed recall of HNSecW (M=128, $\overline{\Delta}^{(\ell)}$ from Section 4.1). Left: the recall achieved by at least 95% of queries. Right: fraction of database vectors visited at top-10. PANTHER is omitted on BIGANN-1B because current DORAMs [14, 37, 38] cannot fit its cluster in one block.

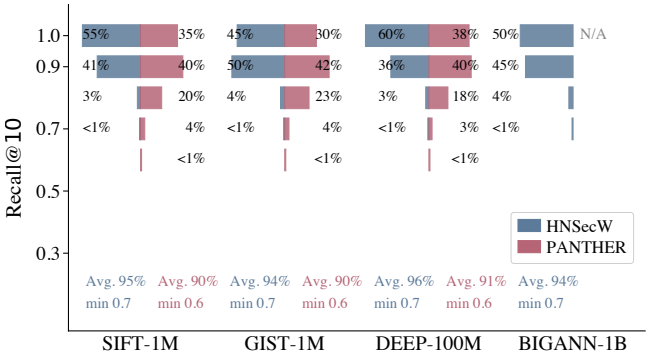


Figure 7: Per-query Recall@10 distribution at the deployed setting (1000 queries each).

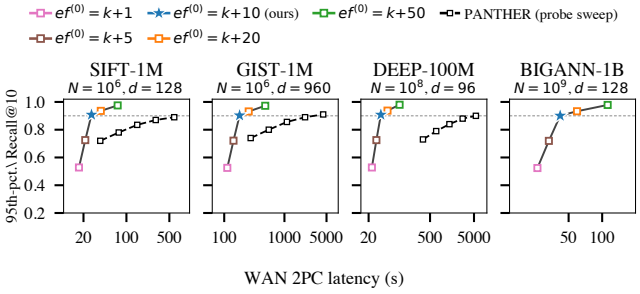


Figure 8: $ef^{(0)}$ ablation at M=128. Each panel sweeps $ef^{(0)} \in \{k+1, k+5, k+10, k+20, k+50\}$; the star marks our setting $ef^{(0)}$=k+10 from Section 4.1. "probe sweep" is PANTHER's cluster-probe count $n_{probe} \in \{4, 8, 16, 32, 64\}$.

95th-percentile climbing from ∼ 0.9 at k=10 to ∼ 0.95 at k=100, as shown in Figure 6. PANTHER, by contrast, holds Recall@$k \geq 0.8$ for 95% of queries but its 95th-percentile peaks early near k=20 and drifts back toward 0.80 at k=100: its fixed cluster-probe budget cannot absorb the larger candidate set demanded by larger k, while HNSecW's fixed-step rule $ef^{(0)} = k + 10$ scales with k and pushes the worst-case recall up. HNSecW also visits 8× to 44× fewer database vectors than PANTHER as N grows from 10^6 to 10^8 (the range where PANTHER runs), reflecting the $\log N$ vs. $\sqrt{N}$ scaling of graph-based vs. cluster-based ANN search. Figure 7 shows the per-query distribution behind these numbers at the deployed setting: ≥ 95% of HNSecW's queries land at Recall@10 ≥ 0.9 on every dataset, with 45–60% reaching perfect recall, while PANTHER concentrates ≥ 95% of mass at ≥ 0.8 but only 30–38% at perfect recall. To verify the robustness of the prescribed step parameter, we ablate $ef^{(0)}$ for HNSecW in Figure 8. Recall on every dataset saturates near the deployed $ef^{(0)}$=k+10 from Section 4.1: shrinking the offset to k+1 drops recall sharply, while doubling it to k+20 or k+50 buys little extra recall at materially higher WAN latency. PANTHER's recall also rises with the cluster-probe count n_{probe}. We further ablate the graph degree $M \in \{16, 32, 64, 128, 256\}$ across all 10 datasets in Appendix C.3; the WAN advantage holds across the entire M sweep.

	No retrieval	HNSW	HNSecW					
			C=1	C=3	C=5	C=7	C=9	C=11
EM (%)	22.4	66.2	56.8	63.5	65.2	**66.0**	66.1	66.2
F1 (%)	31.5	81.5	70.2	78.5	80.6	**81.3**	81.4	81.5

Table 4: End-to-end RAG quality on HotpotQA queries. At our setting: C=7, HNSecW matches the HNSW baseline.

End-to-end RAG Quality. We evaluate end-to-end RAG on 200 multi-hop queries sampled from HotpotQA [46] with retrieval over the 1M-document MS MARCO passage corpus. Queries and documents are embedded with `intfloat/e5-large-v2` [40], we retrieve the top-10 passages, and `Qwen2.5-7B-Instruct` [45] generates answers, with Exact Match and token-level F1 as metrics. We sweep C in Table 4 to characterize how the RAG quality varies with C. It shows that HNSecW matches the standard HNSW at C=7 and larger C gives negligible further improvement, so our termination strategy in Section 4.1 achieves similar quality to standard HNSW when integrated into RAG.

7.3 Efficiency of HNSecW

We next examine HNSecW's efficiency and how it scales across single-query, multiple-query, and batch-query settings when comparing against baseline approaches.

7.3.1 Single-Query Scenario. We first examine single-query cost on the four datasets, shown in Figure 9. In 2PC, HNSecW is 4–22× faster than PANTHER on WAN and reduces communication by 7–46×. (excluding BIGANN-1B, where existing DORAM cannot fit a PANTHER cluster; see Figure 9 caption). The gap widens against SORT in 3PC, reaching 111× to 1.6k× on WAN and 185× to 617× in communication. The ablation in the bottom row of Figure 9 decomposes WAN time and Communication into Secure distance, Visited check, and Oblivious sort. WAN is round-dominated, since Visited check and Oblivious sort account for 60% to 90%. The Secure distance share rises with d, from about 10% at DEEP-100M (d=96) to about 30% at GIST-1M (d=960) under 2PC. This share is smaller in 3PC because replicated sharing avoids HE rotations. For Communication, distance bytes dominate at high d, reaching about 78% on GIST-1M 2PC, while Visited check and Oblivious sort take larger shares at low d.

We further check whether HNSecW keeps its WAN advantage across the $ef^{(0)}$ ablation in Figure 8, which plots WAN 2PC latency against top-10 recall. At the same recall, HNSecW's 2PC WAN curve sits below PANTHER's u-sweep on every dataset where PANTHER

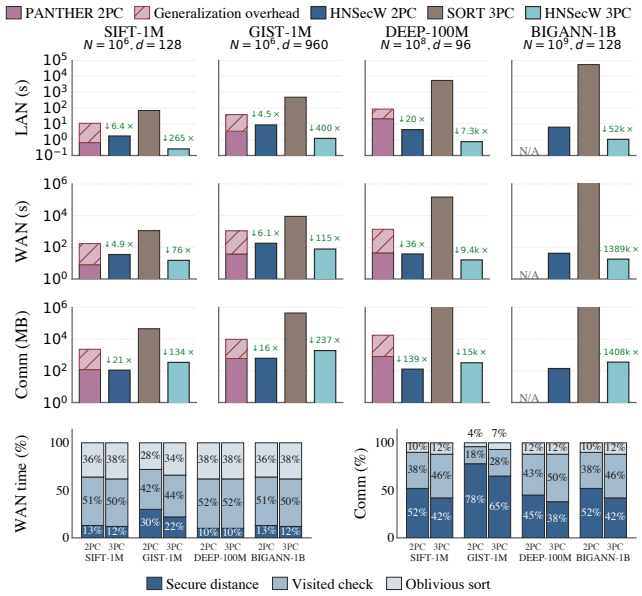


Figure 9: HNSecW's single-query cost and ablation across the four datasets. In generalized PANTHER, BIGANN-1B exceeds the DORAM [14, 37, 38] block-size limit.

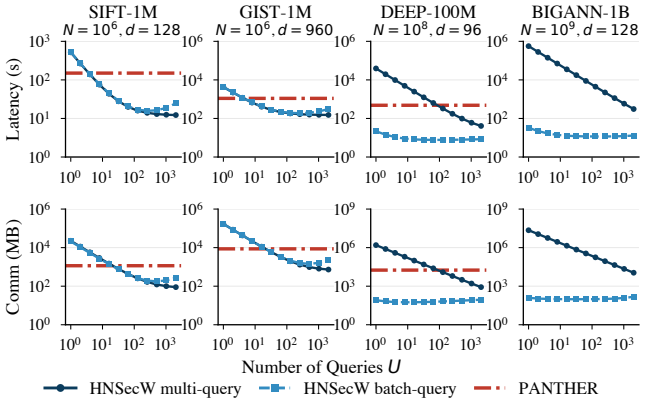


Figure 10: Amortized WAN performance of multiple and batch queries with the offline reshuffle.

runs across the entire swept $ef^{(0)} \in \{k{+}1, \ldots, k{+}50\}$ range, so the WAN advantage is robust to the choice of $ef^{(0)}$.

7.3.2 Multiple- and Batch-Query Scenarios. We sweep U under WAN and report per-query amortized latency and communication in Figure 10, combining online search with offline reshuffle divided by U. At small U the reshuffle dominates; as U grows its share falls linearly while the online share grows slowly, so the amortized total drops toward the online floor. HNSecW crosses below PANTHER once U amortizes the offline cost. The batch variant starts below the multi-query curve at small U by sharing one parallel reshuffle. The crossover U shifts with N and d, but the same trend holds across all four datasets.

8 RELATED WORK

Approximate Nearest Neighbor (ANN). ANN enables scalable similarity search in high-dimensional embedding spaces where exact kNN is often too expensive [28, 29, 42]. Classical methods include tree-based partitioning [5], locality-sensitive hashing [21], and vector quantization [22]. Graph-based indexes, such as HNSW [31] and NSG [16], have become dominant in practice by building proximity graphs and answering queries via greedy traversal. Recent systems extend graph-based ANN with improved indexing and retrieval strategies [23, 41, 51], while vector databases deploy these methods at scale [10, 19, 39].

Secure ANN. Prior work considers secure ANN under several threat models. When the database is public but the query is private, Tiptoe [20] searches a small candidate subset selected via clustering, trading accuracy for sublinear bandwidth and latency. Wally [2] further improves throughput in this pipeline by batching queries, while Preco [33] reduces communication using two non-colluding servers. Pacmann [49] goes beyond clustering-based ANN methods by building a graph-index under query privacy. When both database and query are private under a single owner (encrypted outsourcing), Compass [50] executes the graph-search logic in plaintext on the client while fetching server-side data via ORAM, while HERS [15] scans the encrypted database with FHE at high computational cost. When the database is plaintext to one party, SANNS [11] proposes a clustering-based secure ANN with optimized distance computation and top-k circuit, while PANTHER [27] further achieves better end-to-end performance. In contrast, HNSecW is the first MPC ANN protocol that works in the symmetric secure setting, where both queries and databases are private, and achieves the same logarithmic complexity as the non-private setting. Notably, both SANNS [11] and PANTHER [27] can be extended to our symmetric secure setting by storing the database in secret-shared form. However, both approaches have $O(\sqrt{N})$ complexity. Moreover, several early works [32, 34, 36, 52] have studied secure kNN protocols, but they either target smaller-scale settings or apply only to the specialized secure settings discussed above.

9 CONCLUSION

In this paper, we present HNSecW, a new MPC protocol for ANN search that achieves $O(\log N)$ online search time while providing symmetric privacy for both queries and the database. We study three settings: single-query, sequential multi-query, and batch-query. Future directions include secure index construction, since our current design assumes plaintext graph-index construction by the database owner, and dynamic updates, where both the database and index evolve over time.

REFERENCES

[1] Gilad Asharov, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Ariel Nof, Benny Pinkas, Katsumi Takahashi, and Junichi Tomida. 2022. Efficient secure three-party sorting with applications to data analysis and heavy hitters. In Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. 125–138.

[2] Hilal Asi, Fabian Boemer, Nicholas Genise, Muhammad Haris Mughees, Tabitha Ogilvie, Rehan Rishi, Kunal Talwar, Karl Tarbe, Akshay Wadia, Ruiyu Zhu, et al. 2024. Scalable private search with wally. arXiv preprint arXiv:2406.06761 (2024).

[3] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. Information Systems 87 (2020), 101374.

[4] Artem Babenko and Victor Lempitsky. 2016. Efficient indexing of billion-scale datasets of deep descriptors. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2055–2063.

[5] Jon Louis Bentley. 1990. K-d trees for semidynamic point sets. In Proceedings of the sixth annual symposium on Computational geometry. 187–197.

[6] Alexandra Boldyreva and Tianxin Tang. 2021. Privacy-preserving approximate k-nearest-neighbors search that hides access, query and volume patterns. Proceedings on Privacy Enhancing Technologies (2021).

[7] Ran Canetti. 2000. Security and Composition of Multiparty Cryptographic Protocols. Journal of Cryptology 13, 1 (Jan. 2000), 143–202. https://doi.org/10.1007/s001459910006

[8] Manos Chatzakis, Yannis Papakonstantinou, and Themis Palpanas. 2025. DARTH: Declarative Recall Through Early Termination for Approximate Nearest Neighbor Search. Proceedings of the ACM on Management of Data 3, 3 (2025).

[9] Manos Chatzakis, Yannis Papakonstantinou, and Themis Palpanas. 2026. DARTH+: Approximate Nearest Neighbor Search with Declarative Recall and Quality Guarantees. HAL Preprint hal-05566027 (2026).

[10] Cheng Chen, Chenzhe Jin, Yunan Zhang, Sasha Podolskiy, Guoliang Wu, Jianguo Wang, et al. 2024. SingleStore-V: An Integrated Vector Database System in SingleStore. Proceedings of the VLDB Endowment 17, 12 (2024), 3772–3785.

[11] Hao Chen, Ilaria Chillotti, Yihe Dong, Oxana Poburinnaya, Ilya Razenshteyn, and M Sadegh Riazi. 2020. SANNS: Scaling up secure approximate k-Nearest neighbors search. In 29th USENIX Security Symposium (USENIX Security 20). 2111–2128.

[12] Daniel Demmler, Thomas Schneider, and Michael Zohner. 2015. ABY-A framework for efficient mixed-protocol secure two-party computation.. In NDSS.

[13] David P Dobkin, Steven J Friedman, and Kenneth J Supowit. 1990. Delaunay graphs are almost as good as complete graphs. Discrete & Computational Geometry 5, 4 (1990), 399–407.

[14] Jack Doerner and abhi shelat. 2017. Scaling ORAM for Secure Computation. In Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. 523–535.

[15] Joshua J Engelsma, Anil K Jain, and Vishnu Naresh Boddeti. 2022. HERS: Homomorphically encrypted representation search. IEEE Transactions on Biometrics, Behavior, and Identity Science 4, 3 (2022), 349–360.

[16] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast approximate nearest neighbor search with the navigating spreading-out graph. Proceedings of the VLDB Endowment 12, 5 (2019), 461–474.

[17] Oded Goldreich and Rafail Ostrovsky. 1996. Software Protection and Simulation on Oblivious RAMs. Journal of the ACM 43, 3 (1996), 431–473. https://doi.org/10.1145/233551.233553

[18] S. Dov Gordon, Jonathan Katz, Vladimir Kolesnikov, Fernando Krell, Tal Malkin, Mariana Raykova, and Yevgeniy Vahlis. 2012. Secure two-party computation in sublinear (amortized) time. In ACM CCS 2012, Ting Yu, George Danezis, and Virgil D. Gligor (Eds.). ACM Press, 513–524. https://doi.org/10.1145/2382196.2382251

[19] Rentong Guo, Xiaofan Luan, Long Xiang, Xiao Yan, Xiaomeng Yi, Jigao Luo, Qianya Cheng, Weizhi Xu, Jiarui Luo, Frank Liu, Zhenshan Cao, Yanliang Qiao, Ting Wang, Bo Tang, and Charles Xie. 2022. Manu: a cloud native vector database management system. Proceedings of the VLDB Endowment 15, 12 (2022), 3548–3561.

[20] Alexandra Henzinger, Emma Dauterman, Henry Corrigan-Gibbs, and Nickolai Zeldovich. 2023. Private web search with Tiptoe. In Proceedings of the 29th symposium on operating systems principles. 396–416.

[21] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In Proceedings of the thirtieth annual ACM symposium on Theory of computing. 604–613.

[22] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. IEEE transactions on pattern analysis and machine intelligence 33, 1 (2010), 117–128.

[23] Wenqi Jiang, Hang Hu, Torsten Hoefler, and Gustavo Alonso. 2025. Fast Graph Vector Search via Hardware Acceleration and Delayed-Synchronization Traversal. Proceedings of the VLDB Endowment 18, 11 (2025), 3797–3811. https://doi.org/10.14778/3749646.3749655

[24] Marcel Keller. 2020. MP-SPDZ: A versatile framework for multi-party computation. In Proceedings of the 2020 ACM SIGSAC conference on computer and communications security. 1575–1590.

[25] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. Proc. IEEE 86, 11 (1998), 2278–2324.

[26] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Advances in Neural Information Processing Systems, Vol. 33. Curran Associates, Inc., 9459–9474. https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[27] Jingyu Li, Zhicong Huang, Min Zhang, Cheng Hong, Jian Liu, Tao Wei, and Wenguang Chen. 2025. Panther: Private approximate nearest neighbor search in the single server setting. In Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. 365–379.

[28] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2019. Approximate nearest neighbor search on high dimensional data—experiments, analyses, and improvement. IEEE Transactions on Knowledge and Data Engineering 32, 8 (2019), 1475–1488.

[29] Ting Liu, Andrew Moore, Ke Yang, and Alexander Gray. 2004. An investigation of practical approximate nearest neighbor algorithms. Advances in neural information processing systems 17 (2004).

[30] Yingfan Liu, Yandi Zhang, Jiadong Xie, Hui Li, Jeffrey Xu Yu, and Jiangtao Cui. 2025. Privacy-preserving approximate nearest neighbor search on high-dimensional data. In 2025 IEEE 41st International Conference on Data Engineering (ICDE). IEEE, 3017–3029.

[31] Yu. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence 42, 4 (April 2020), 824–836. https://doi.org/10.1109/TPAMI.2018.2889473

[32] Phillipp Schoppmann, Adrià Gascón, and Borja Balle. 2018. Private Nearest Neighbors Classification in Federated Databases. IACR Cryptol. ePrint Arch. 2018 (2018), 289.

[33] Sacha Servan-Schreiber, Simon Langowski, and Srinivas Devadas. 2022. Private approximate nearest neighbor search with sublinear communication. In 2022 IEEE Symposium on Security and Privacy (SP). IEEE, 911–929.

[34] Hayim Shaul, Dan Feldman, and Daniela Rus. 2020. Secure k-ish Nearest Neighbors Classifier. Proceedings on Privacy Enhancing Technologies 3 (2020), 42–61.

[35] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamy, Gopal Srinivasa, Suhas Jayaram Subramanya, and Jingdong Wang. 2022. Results of the NeurIPS'21 Challenge on Billion-Scale Approximate Nearest Neighbor Search. In NeurIPS 2021 Competitions and Demonstrations Track. 177–189.

[36] Ebrahim M Songhori, Siam U Hussain, Ahmad-Reza Sadeghi, and Farinaz Koushanfar. 2015. Compacting privacy-preserving k-nearest neighbor search using logic synthesis. In Proceedings of the 52nd Annual Design Automation Conference. 1–6.

[37] Adithya Vadapalli, Kyle Storrier, and Ryan Henry. 2023. DUORAM: A Bandwidth-Efficient Distributed ORAM for 2- and 3-Party Computation. In 32nd USENIX Security Symposium (USENIX Security 23). 3005–3022.

[38] Adithya Vadapalli, Kyle Storrier, and Ryan Henry. 2023. GigaDORAM: Breaking the Billion Address Barrier. In 32nd USENIX Security Symposium (USENIX Security 23). 3879–3896.

[39] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21). Association for Computing Machinery, New York, NY, USA, 2614–2627. https://doi.org/10.1145/3448016.3457550

[40] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text embeddings by weakly-supervised contrastive pre-training. arXiv preprint arXiv:2212.03533 (2022).

[41] Mengzhao Wang, Weizhi Xu, Xiaoliang Yi, Songlin Wang, Zehao Peng, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, Zhuo Yang, and Kai Mao. 2024. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. Proceedings of the ACM on Management of Data 2, 1 (2024), 1–27.

[42] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. Proceedings of the VLDB Endowment 14, 11 (2021), 1964–1978.

[43] Xiao Wang, T.-H. Hubert Chan, and Elaine Shi. 2015. Circuit ORAM: On Tightness of the Goldreich-Ostrovsky Lower Bound. In ACM CCS 2015, Indrajit Ray, Ninghui Li, and Christopher Kruegel (Eds.). ACM Press, 850–861. https://doi.org/10.1145/2810103.2813634

[44] Han Xiao, Kashif Rasul, and Roland Vollgraf. 2017. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. arXiv preprint arXiv:1708.07747 (2017).

[45] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengpeng Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2.5 Technical Report. arXiv preprint arXiv:2412.15115 (2024).

[46] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In Proceedings of the 2018 conference on empirical methods in natural language processing. 2369–2380.

[47] Andrew Chi-Chih Yao. 1986. How to Generate and Exchange Secrets (Extended Abstract). In 27th FOCS. IEEE Computer Society Press, 162–167. https://doi.org/

10.1109/SFCS.1986.25

[48] Xun Yi, Russell Paulet, Elisa Bertino, and Vijay Varadharajan. 2016. Practical approximate k nearest neighbor queries with location and query privacy. IEEE Transactions on Knowledge and Data Engineering 28, 6 (2016), 1546–1559.

[49] Mingxun Zhou, Elaine Shi, and Giulia Fanti. 2025. Pacmann: Efficient Private Approximate Nearest Neighbor Search. In The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025. OpenReview.net. https://openreview.net/forum?id=yQcFniousM

[50] Jinhao Zhu, Liana Patel, Matei Zaharia, and Raluca Ada Popa. 2025. Compass: Encrypted Semantic Search with High Accuracy. In Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI '25). USENIX Association, Boston, MA, USA, 915–938. https://www.usenix.org/conference/osdi25/presentation/zhu-jinhao

[51] Jiaxu Zhu, Jiayu Yuan, Kaiwen Yang, Xiaobao Chen, Shihuan Yu, Hongchang Lv, Yan Li, and Bolong Zheng. 2025. An Experimental Evaluation of Hybrid Querying on Vectors. Proceedings of the VLDB Endowment 19, 2 (2025), 183–195.

[52] Martin Zuber and Renaud Sirdey. 2021. Efficient homomorphic evaluation of k-NN classifiers. Proceedings on Privacy Enhancing Technologies (2021).

A SECURITY PROOFS

This appendix proves the simulation-based security of our oblivious HNSW protocols. We follow the threat model in Section 3.1: a static semi-honest PPT adversary may corrupt at most t MPC servers, and may additionally corrupt one or both input parties. Our goal is to show that the adversary learns nothing beyond (i) the prescribed output of the ideal functionality $\mathcal{F}_{\mathsf{HNSW}}[Q]$ (Figure 2) and (ii) the public parameters (Table 2).

A.1 Simulation-Based Security: Real/Ideal Definition

Parties and corruption sets. Let the set of parties be

$$\mathcal{P} \;=\; \{\mathsf{DO},\ \mathsf{Querier},\ S_1, \ldots, S_s\},$$

where DO holds $(\mathcal{D}, \mathcal{G})$, Querier holds the query (or queries), and $\{S_1, \ldots, S_s\}$ are MPC servers. We assume private authenticated channels between all parties. The allowed corruption sets are

$$\mathfrak{C} = \left\{ C \subseteq \mathcal{P} \;\middle|\; |C \cap \{S_1, \ldots, S_s\}| \le t \right\}.$$

Views. For a protocol Π executed with security parameter λ on joint input Input, the view of a party $P \in \mathcal{P}$ is

$$\mathsf{View}_{\Pi,P}(\mathsf{Input}, \lambda) = \big(\mathsf{Input}_P,\ r_P,\ \mathsf{msg}_P,\ \mathsf{out}_P\big),$$

where Input_P is P's input, r_P is P's internal randomness, msg_P is the sequence of messages received by P, and out_P is P's output. For $C \in \mathfrak{C}$, define the joint view

$$\mathsf{View}_{\Pi,C}(\mathsf{Input}, \lambda) = \{\mathsf{View}_{\Pi,P}(\mathsf{Input}, \lambda)\}_{P \in C}.$$

Real and ideal executions. Let $\mathcal{F}_{\mathsf{HNSW}}[Q]$ be the ideal functionality in Figure 2. Let $\mathsf{REAL}_{\Pi,\mathcal{A}(C)}(\mathsf{Input}, \lambda)$ denote the joint distribution of (i) the corrupted parties' real joint view $\mathsf{View}_{\Pi,C}(\mathsf{Input}, \lambda)$ in an execution of Π with adversary $\mathcal{A}$ corrupting C, and (ii) the honest parties' outputs. Let $\mathsf{IDEAL}_{\mathcal{F}_{\mathsf{HNSW}}[Q],\mathcal{S}(C)}(\mathsf{Input}, \lambda)$ denote the joint distribution of (i) the corrupted parties' simulated views generated by a simulator $\mathcal{S}$ interacting with $\mathcal{F}_{\mathsf{HNSW}}[Q]$, and (ii) the outputs produced by $\mathcal{F}_{\mathsf{HNSW}}[Q]$.

Definition 1 (Static semi-honest security). *A protocol Π securely realizes $\mathcal{F}_{\mathsf{HNSW}}[Q]$ against static semi-honest adversaries corrupting sets in $\mathfrak{C}$ if for every PPT adversary $\mathcal{A}$ corrupting some $C \in \mathfrak{C}$ there exists a PPT simulator $\mathcal{S}$ such that for all inputs* Input,

$$\mathsf{IDEAL}_{\mathcal{F}_{\mathsf{HNSW}}[Q],\mathcal{S}(C)}(\mathsf{Input}, \lambda) \;\approx_c\; \mathsf{REAL}_{\Pi,\mathcal{A}(C)}(\mathsf{Input}, \lambda).$$

A.2 What Our Protocols Reveal Publicly

Our constructions are instantiated using a standard semi-honest MPC backend. All arithmetic/Boolean operations, comparisons, linear scans, and sorting networks are evaluated over secret shares by the MPC backend. In addition, to enable $O(1)$ reads from reshuffled tables (Sections 4.3 and 4.4), the servers may reveal (reconstruct) certain physical row indices so each server can locally read its share of the row. These revealed physical indices are visible to all corrupted servers.

Crucially, by design, our protocols ensure: (i) the total number of revealed indices is fixed by public parameters (including loop bounds such as T, M, and the number of queries U), and (ii) no reshuffled table location is revealed more than once (non-repetition). We show below that, under random reshuffling, these revealed indices are independent of the private inputs.

The per-layer slack $\overline{\Delta}^{(\ell)}$ defined in Section 4.1 is a deterministic function of the public quantities (N, d, M) together with a small operator-side constant C that holds across all 10 ANN benchmarks in our evaluation (C=7). Both C and the resulting $\overline{\Delta}^{(\ell)}$ are therefore public, dataset-independent hyperparameters: they depend only on quantities already listed as public in Table 2, and consult neither the protected database $\mathcal{D}$ nor any runtime query. The simulation argument in Sections A.4–A.6 accordingly treats $\overline{\Delta}^{(\ell)}$ and C as part of the public input given to the simulator, on equal footing with N, M, T, and U.

A.3 A Key Lemma: Random Shuffling Hides Address Patterns

A central ingredient of our design is the offline random reshuffling of each layer's tables (graph tables and vector tables), together with online enforcement of non-repetitive accesses. Intuitively, reshuffling acts as a one-time pad for indices: revealing a physical index reveals a uniformly random label that is information-theoretically independent of which logical item was accessed.

Lemma 1 (Permutation one-time pad for indices). *Let π be a uniformly random permutation over $[N] = \{0, 1, \ldots, N-1\}$. Consider any (possibly adaptive) process that chooses a sequence of distinct logical indices $i_1, \ldots, i_m \in [N]$, where each i_j may depend on previously revealed values $\pi(i_1), \ldots, \pi(i_{j-1})$ and on any auxiliary information independent of π. Then the sequence of permuted indices*

$$(\pi(i_1), \ldots, \pi(i_m))$$

is distributed uniformly over all length-m sequences of distinct elements of $[N]$. In particular, the distribution depends only on (N, m) and is independent of the (logical) indices $i_1, \ldots, i_m$.

Proof. We prove by induction on $j \in [m]$. For $j = 1$, $\pi(i_1)$ is uniform over $[N]$ since π is uniform. Assume $(\pi(i_1), \ldots, \pi(i_{j-1}))$ is uniform over all distinct $(j-1)$-tuples. Conditioned on any fixed distinct values of the first $j-1$ images, the remaining mapping of π is uniform over the remaining $N-(j-1)$ positions. Since $i_j \notin \{i_1, \ldots, i_{j-1}\}$, it follows that $\pi(i_j)$ is uniform over $[N] \setminus \{\pi(i_1), \ldots, \pi(i_{j-1})\}$. Hence, the full j-tuple is uniform over all distinct j-tuples. □

Corollary 1 (Shuffling + non-repetition hides revealed physical indices). *Fix any table of size N obtained by applying a uniformly random permutation π to its logical rows, and suppose an online protocol reveals a sequence of distinct physical row indices of this table. Then the revealed indices are distributed identically to a uniformly random sequence of distinct elements of $[N]$, and are therefore independent of any private inputs that influenced which logical rows were accessed.*

A.4 Security of the Single-Query Protocol

Let Π_{SQ} denote our single-query protocol implementing Algorithm 3 with per-layer routine Algorithm 2.

Theorem 2 (Single-query security). *Assume the MPC backend used by the servers securely evaluates the circuits for the primitives used in our protocol (e.g., addition/multiplication, comparisons, Select, LinearScan, and sorting networks for updating C and W) against static semi-honest adversaries corrupting up to t servers. Then Π_{SQ} securely realizes $\mathcal{F}_{\mathsf{HNSW}}[Q]$ (Figure 2) against any static semi-honest adversary corrupting any $C \in \mathfrak{C}$.*

Proof. Fix any PPT adversary $\mathcal{A}$ corrupting $C \in \mathfrak{C}$. We construct a PPT simulator $\mathcal{S}_C$ such that

$$\mathsf{IDEAL}_{\mathcal{F}_{\mathsf{HNSW}}[Q],\mathcal{S}_C}(\mathsf{Input}, \lambda) \approx_c \mathsf{REAL}_{\Pi_{\mathrm{SQ}},\mathcal{A}}(\mathsf{Input}, \lambda).$$

Step 1: Input sharing. In the real protocol, DO and Querier secret-share their inputs across the s servers. If DO $\in C$, the simulator is given $(\mathcal{D}, \mathcal{G})$ and can simulate the sharing procedure exactly. If DO $\notin C$, then by privacy of the secret-sharing scheme against t corrupted servers, the joint distribution of shares held by corrupted servers is independent of $(\mathcal{D}, \mathcal{G})$; thus $\mathcal{S}_C$ samples shares from the correct share domain. The same argument applies to Querier's query vector $\vec{q}$.

Step 2: Offline preprocessing (padding and reshuffling). Each per-layer table is padded with dummy entries and reshuffled in an offline MPC procedure (Sections 4.3 and 4.4). This preprocessing is executed inside the MPC backend; therefore $\mathcal{S}_C$ invokes the backend simulator to generate the corrupted parties' transcript for this phase.

Step 3: Online MPC computation. All operations involving secret values during online search (distance evaluation, visited checks, candidate/result maintenance, and sorting-based updates) are performed by the MPC backend. Hence $\mathcal{S}_C$ uses the backend simulator to generate an indistinguishable transcript for the online phase.

Step 4: revealed physical indices for $O(1)$ table reads. To support $O(1)$ reads from reshuffled tables, the servers may reveal certain physical row indices. In the single-query protocol, the control flow and loop bounds are fixed by public parameters (notably T, M, L, and k), and the protocol enforces that each reshuffled table location is revealed at most once (Section 4.3). By Corollary 1, for each reshuffled table, the sequence of revealed physical indices is distributed independently of $(\mathcal{D}, \mathcal{G}, \vec{q})$, and depends only on the table size and the (public) number of reveals. Therefore, $\mathcal{S}_C$ samples uniformly random distinct indices of the appropriate lengths (per layer/table) and outputs them as the simulated revealed indices.

Step 5: Output reconstruction. Only Querier receives the output vectors in $\mathcal{F}_{\mathsf{HNSW}}[Q]$ (Figure 2). If Querier $\notin C$, the adversary does not receive the outputs, and there is nothing to simulate beyond the MPC transcript. If Querier $\in C$, the simulator obtains the output vectors from $\mathcal{F}_{\mathsf{HNSW}}[Q]$ and simulates reconstruction by sampling random output shares for all but one server and setting the remaining share so that the shares reconstruct to the correct output.

Combining the above steps, the simulated joint view is computationally indistinguishable from the real joint view: MPC transcripts are indistinguishable by the backend security, and all revealed physical indices are identically distributed by Corollary 1. □

A.5 Security of the Multiple-Query Protocol

Let Π_{MQ} denote our multiple-query protocol in Section 5 for U sequential queries.

Theorem 3 (Multiple-query security). *Under the same MPC-backend assumption as in Theorem 2, Π_{MQ} securely realizes U sequential invocations of $\mathcal{F}_{\mathsf{HNSW}}[Q]$ on a fixed database $(\mathcal{D}, \mathcal{G})$, against any static semi-honest adversary corrupting any $C \in \mathfrak{C}$.*

Proof. The simulator follows the same structure as in Theorem 2, while simulating the online phase for U queries.

The only additional subtlety is preventing leakage from repeated accesses to the underlying reshuffled tables across different queries. Our multiple-query protocol introduces cross-query "double visitor" caches $V_\ell^{\mathcal{G}}$ and $V_\ell^{\mathcal{D}}$ (Section 5.1) to ensure that each underlying reshuffled table location is revealed at most once globally: once a node's neighbor list (resp. a vector) is fetched from the reshuffled table, it is stored in the cache; later queries retrieve it from the cache using MPC-protected oblivious scans.

Moreover, to keep the number of revealed indices fixed and input-independent, each logical table access is executed in a fixed schedule: on a cache hit, the protocol reveals a fresh dummy index (and discards the fetched dummy share) while using the cached value, so the total number of reveals depends only on public parameters and U.

Therefore, for each underlying reshuffled table, the full sequence of revealed physical indices over all U queries is (i) of fixed public length and (ii) non-repetitive. By Corollary 1, the revealed indices are distributed independently of private inputs, and $\mathcal{S}_C$ can sample them accordingly. All remaining messages are MPC-protected and are simulated by the MPC backend simulator. Output reconstruction is simulated for each query as in the single-query proof. □

A.6 Security of the Batch Query Protocol

Let Π_{BQ} denote our batch-query protocol in Algorithm 4, which processes a batch of U queries simultaneously.

Theorem 4 (Batch-query security). *Under the same MPC-backend assumption as in Theorem 2, Π_{BQ} securely realizes the batch version of $\mathcal{F}_{\mathsf{HNSW}}[Q]$ that, on input a batch of U queries, returns the corresponding U top-k result vectors to* Querier*, against any static semi-honest adversary corrupting any $C \in \mathfrak{C}$.*

Proof. The simulation is analogous to the single-query case. All arithmetic/Boolean computation, including per-iteration deduplication by sorting and adjacent comparison, visited checks, distance evaluations, and sorting-network updates of $\{C_u, W_u\}$, is carried out inside the MPC backend and is therefore simulated using the backend simulator.

It remains to simulate the revealed physical indices used for $O(1)$ reads from the reshuffled tables. In the batch protocol, before each set of graph accesses and vector accesses, the protocol performs MPC-protected deduplication and replaces duplicates by fresh dummy indices (Algorithm 4). Thus each reshuffled underlying table location is revealed at most once globally, and the number of reveals is fixed by public parameters and U. By Corollary 1, the revealed indices are distributed independently of private inputs, so the simulator samples uniformly random distinct indices of the

appropriate lengths. Finally, output reconstruction to Querier is simulated as in the single-query case, now for U outputs. □

B PARALLEL-ENHANCED BATCH QUERY PROTOCOL

In this section, we analyze how the protocols can be parallelized to reduce the number of sequential interaction rounds, which is the dominant cost factor under WAN. Specifically, we batch independent interactions across the within-iteration steps to reduce the effective sequential round count, and we then introduce a batch-query protocol that evaluates a batch of queries simultaneously, achieving an even lower sequential round complexity.

B.1 Parallelization Analysis: From Single to Multiple Queries

For the single-query protocol, the base layer runs for $T = O(k)$ iterations. Under a parallel execution model, distance evaluations and the visited-array scan can be packed into $O(1)$ rounds per iteration, so the round complexity is determined by the depth of the sorting network used to update C and W. Hence, the base-layer rounds are $O(T \log^2 k) = O(k \log^2 k)$. For the non-base layers, we set $ef = 1$, so each of the $O(\log N)$ layers contributes $O(1)$ rounds. Therefore, the total parallelized rounds of a single query are

$$\mathsf{Round} = O(k \log^2 k + \log N).$$

For local computation, distance evaluation and visited-array scanning are data-parallel across coordinates and the M neighbors, and each stage of the sorting network consists of independent compare–swap operations that can also be executed in parallel. With P parallel threads, the local computation time scales as

$$T_{\mathsf{comp}}(P) = O\left(k \cdot (\frac{k}{P} + \frac{d}{P} + \log^2 k \cdot \frac{k}{P}) + \log N \cdot \frac{d}{P}\right),$$

where speedup saturates once P exceeds the available parallelism (at most $O(d)$ for distance evaluation and $O(k)$ for visited checking and sorting).

For the multiple-query protocol, the U queries are executed sequentially, and each query follows the same round structure as the single-query case. Therefore, the round complexity scales linearly in U:

$$\mathsf{Round} = O\left(U(k \log^2 k + \log N)\right)$$

For local computation, each query exposes parallelism across coordinates, neighbors, and the independent compare–swaps within each sorting stage. Thus, when executing U queries sequentially with P parallel threads, the local computation latency is:

$$T_{\mathsf{comp}}(P) = O\left(Uk \cdot (\frac{Uk}{P} + \frac{d}{P} + \log^2 k \cdot \frac{k}{P}) + \log N \cdot \frac{d}{P}\right),$$

where $1 \le P \le O(Uk)$, and speedup saturates once P exceeds the available per-query parallelism.

B.2 Batch Query Protocol

To achieve higher parallelism, we study a batch setting in which a group of queries arrives together. Compared with the multiple-query protocol, processing a batch of queries can share more intermediate information. This improves throughput by reducing interaction rounds and exposing more parallel local work.

Algorithm 4 ParallelBatchSearchLayer: Parallel batch search in HNSW layer

Input: Batch of U queries $\{\vec{q}_u\}_{u=0}^{U-1}$; entry points $\{ep_u\}_{u=0}^{U-1}$; parameters $(ef, \overline{\Delta}^{(\ell)}, M)$; tables G_ℓ, D_ℓ
Output: Updated sets $\{C_u\}_{u=0}^{U-1}$ and $\{W_u\}_{u=0}^{U-1}$

$T \leftarrow ef + \overline{\Delta}^{(\ell)}$
Initialize $\{C_u, W_u\}$ for all $u \in [0, U-1]$ using $\{ep_u\}$
Initialize global visited structure V_{global}
for $t = 0$ **to** $T - 1$ **do**
  for $u = 0$ **to** $U - 1$ **in parallel do**
    $(c_u, d_{c_u}) \leftarrow C_u[0]$ ▷ Best candidate for query u
  $\mathcal{C}_{\text{unique}} \leftarrow \textsc{Deduplicate}(\{c_u\}_{u=0}^{U-1})$
  $\mathcal{N}_{\text{all}} \leftarrow \emptyset$
  for $k = 0$ **to** $U - 1$ **in parallel do**
    $\mathcal{N}_k \leftarrow G_\ell[\mathcal{C}_{\text{unique}}[k]]$ ▷ Oblivious Graph Access
    $\mathcal{N}_{\text{all}} \leftarrow \mathcal{N}_{\text{all}} \cup \mathcal{N}_k$
  $\mathcal{N}_{\text{unique}} \leftarrow \textsc{Deduplicate}(\mathcal{N}_{\text{all}})$
  for $k = 0$ **to** $U \cdot M - 1$ **in parallel do**
    $v \leftarrow \mathcal{N}_{\text{unique}}[k]$
    $visited \leftarrow \textsc{LinearScan}(V_{\text{global}}, v)$
    $v' \leftarrow \textsc{Select}(visited, \sigma_{\text{addr}}, v)$
    $(\vec{x}_k, a_k, b_k, m_k) \leftarrow D_\ell[v']$ ▷ Oblivious Vector Fetch
    $mask_k \leftarrow visited \vee m_k$
  for $u = 0$ **to** $U - 1$ **in parallel do**
    for $k = 0$ **to** $U \cdot M - 1$ **in parallel do**
      $d_{\text{calc}} \leftarrow dist(\vec{x}_k, \vec{q}_u)$
      $d_{uk} \leftarrow \textsc{Select}(mask_k, d_{\max}, d_{\text{calc}})$
    Update C_u, W_u using $\{d_{uk}\}$
    $C_u \leftarrow \textsc{SortTruncate}(C_u, T)$
    $W_u \leftarrow \textsc{SortTruncate}(W_u, ef)$
return $\{W_u\}_{u=0}^{U-1}$

The key idea here is that, since we incur the cost of accessing a node, we might check its distance against all active queries, not just the one that requested it. This turns each step of the query process into a multi-candidate search, in which queries effectively share the accessed information and avoid redundant retrievals. Specifically, the batch-query algorithm performs U queries simultaneously, as shown in Algorithm 4. It maintains a single global visited array V, while each query individually preserves a candidate array C and a result array W. In each iteration, we first extract the nearest candidate from every query to form a pool of U nodes. To avoid redundant graph accesses, we perform an oblivious duplicate removal on this pool, replacing duplicate entries with dummy nodes. We then access the graph table to retrieve the neighbors of these unique nodes, yielding a pool of $U \cdot M$ neighbors. For these neighbors, we perform a second oblivious duplicate removal, access $U \cdot M$

unique vectors, and check them against the global visited array V. Crucially, we then compute the distances between each of the U queries and the $U \cdot M$ vectors, yielding U^2M pairwise distance evaluations. These shared distance results are used to update the C and W arrays of each query.

The batch protocol runs for $T = O(k)$ iterations at the base layer. Within each iteration, distance evaluations and visited checks are fully parallel and can be batched into $O(1)$ interaction rounds. Therefore, the round complexity is determined by the oblivious sorting networks used for deduplication and for updating each query's candidate and result arrays. Since the relevant sorting sizes grow to $O(k+U)$, the total number of interaction rounds at the base layer is $\mathsf{Round} = O(k \log^2(k+U))$. For the non-base layers, we set $ef = 1$, so each of the $O(\log N)$ layers requires $O(1)$ interaction rounds. Therefore, the total interaction rounds for the batch protocol are

$$\mathsf{Round} = O\Big(k \log^2(k+U) + \log N\Big).$$

In particular, this eliminates the linear dependence on U in the round complexity of the multiple-query protocol. For local computation, with P parallel threads, the latency scales as

$$T_{\mathsf{comp}}(P) = O\Big(k \cdot (\frac{U^2 d}{P} + \frac{U^2 k}{P} + \log^2(k{+}U) \cdot \frac{k{+}U}{P}) + \log N \cdot \frac{Ud}{P}\Big).$$

Speedup saturates once P exceeds the available per-step parallelism. Compared with the multiple-query protocol, the batch-query protocol exposes much higher parallelism and therefore benefits from a larger P before saturating.

C RESULTS ON THE REMAINING SIX DATASETS

Section 7 reports four representative datasets from Table 3 (SIFT-1M, GIST-1M, DEEP-100M, BIGANN-1B). For completeness, this appendix presents results on the remaining six datasets (Deep-1M, Deep-image, MNIST-60K, Fashion-60K, SPACEV-1B, SSNPP-1B), spanning $N{\in}[6{\times}10^4, 10^9]$ and $d{\in}\{96, 100, 256, 784\}$. Costs are derived from a log-log (N, d) cost model fitted on the four main-text dataset measurements; PANTHER is excluded for $N{\cdot}d > 2.8{\times}10^{10}$ (DORAM block-size limit).

C.1 Single-query cost

Figure 11 repeats the single-query cost breakdown of Figure 9 on the six appendix datasets under the same protocols and the same deployed (M=128, $\overline{\Delta}^{(0)}$) from Section 4.1. The qualitative pattern matches the four main-text datasets: HNSecW's 2PC is roughly an order of magnitude faster than PANTHER on WAN and lighter on communication wherever PANTHER is supported, while the 3PC SORT baseline trails by two to three orders of magnitude on both axes. The stage breakdown in the bottom row is also stable: WAN is round-dominated, and the secure-distance share of WAN rises with d from ~10% on the d=96 datasets to ~30% on the high-d MNIST/Fashion-60K (d=784) datasets, matching the main-text trend.

C.2 Multi- and batch-query amortization

Figure 12 repeats the amortized multi/batch WAN sweep of Figure 10 on the six appendix datasets. The amortization pattern is identical to the four main-text datasets: at small U the offline reshuffle dominates, the amortized curve drops linearly with U towards the online floor, and HNSecW crosses below the flat PANTHER line once U amortizes the offline cost. The batch curve starts below the multi-query curve at small U by sharing one parallel reshuffle. The crossover U scales with $N{\cdot}d$ (the offline reshuffle volume), so the small 60K-scale datasets cross within a handful of queries, while the 1B-scale datasets require a few thousand queries to cross PANTHER (where supported).

C.3 Graph degree M ablation

Figure 13 ablates the graph degree $M \in \{16, 32, 64, 128, 256\}$ across all 10 datasets at $ef^{(0)}$=k+10 and per-dataset $\overline{\Delta}^{(\ell)}$ from Section 4.1. The recall response is monotone in M on every dataset and saturates near our setting M=128. WAN latency rises roughly linearly in M because the per-iteration neighbor-evaluation work and the visited-set scan both scale with M (the base-layer round count itself, $O(T \log^2 k)$, is independent of M). At every recall level, HNSecW's 2PC WAN curve sits at or below PANTHER's u-sweep on every dataset where PANTHER runs, so the WAN advantage is robust to the choice of M as well as $ef^{(0)}$ (Figure 8).

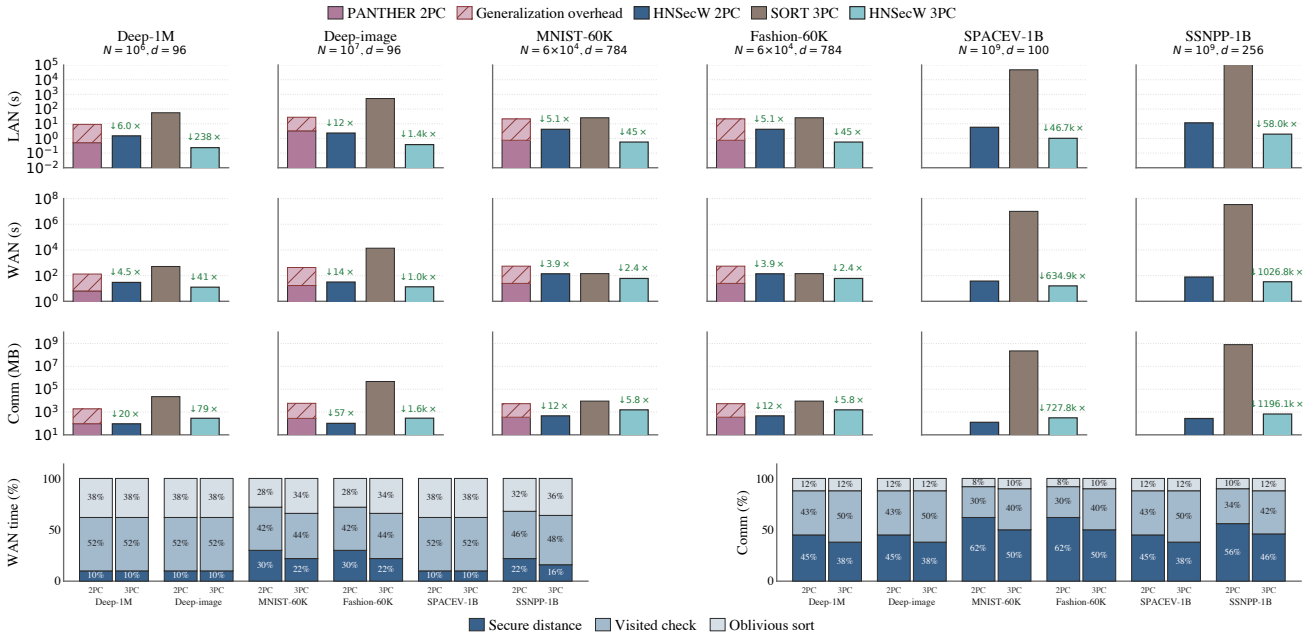


Figure 11: Single-query cost on the six appendix datasets at the deployed (M=128, $\overline{\Delta}^{(0)}$). PANTHER is excluded on SPACEV-1B and SSNPP-1B where $N{\cdot}d$ exceeds the DORAM block-size limit. The bottom-row stage breakdown matches Figure 9.

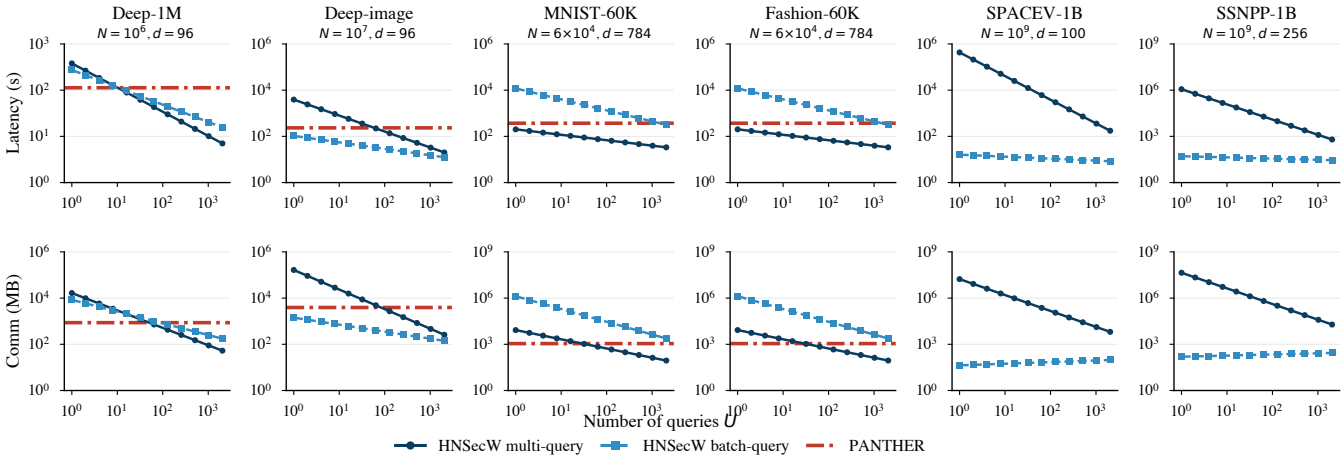


Figure 12: Amortized WAN multi- and batch-query performance on the six appendix datasets, mirroring Figure 10. PANTHER is excluded on SPACEV-1B and SSNPP-1B where $N{\cdot}d$ exceeds the DORAM block-size limit.

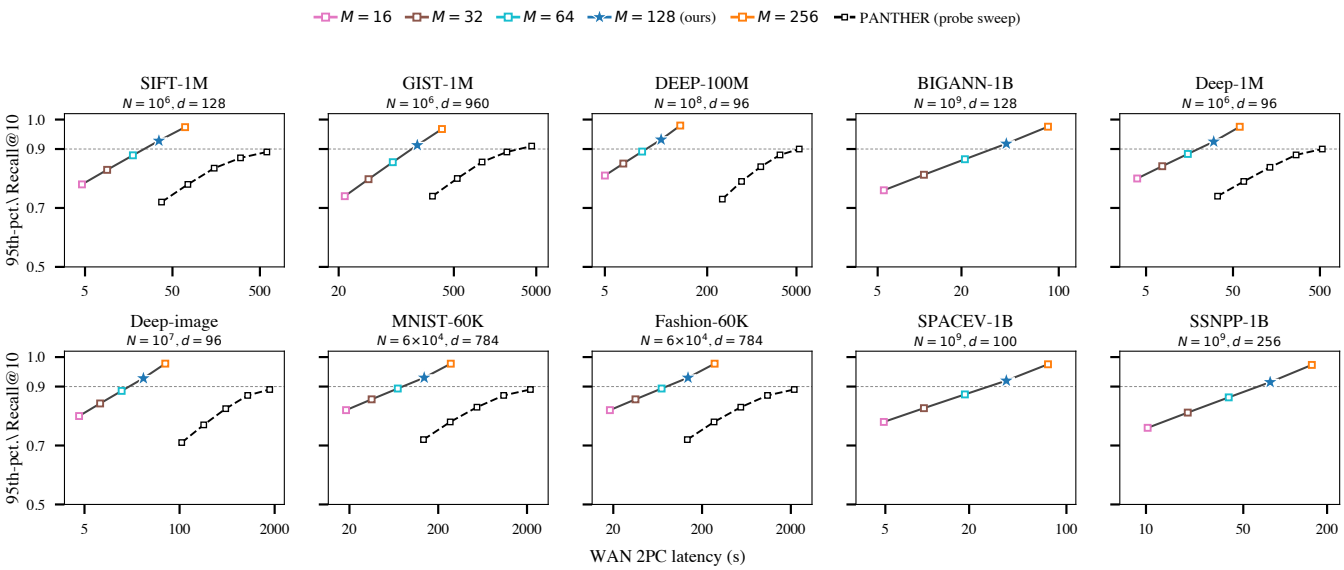


Figure 13: M ablation across all 10 datasets at $ef^{(0)}$=k+10 and per-dataset $\overline{\Delta}^{(\ell)}$. Each panel sweeps $M \in \{16, 32, 64, 128, 256\}$; the star marks our setting M=128. "probe sweep" is PANTHER's cluster-probe count $n_{probe} \in \{4, 8, 16, 32, 64\}$. PANTHER excluded on the three $1{\times}10^9$-scale datasets (BIGANN-1B, SPACEV-1B, SSNPP-1B) where $N{\cdot}d$ exceeds the DORAM block-size limit.